Deep Sequence Modeling: RNN to Transformers

JENNIFER JOYCE M. MONTEMAYOR - MAULANA

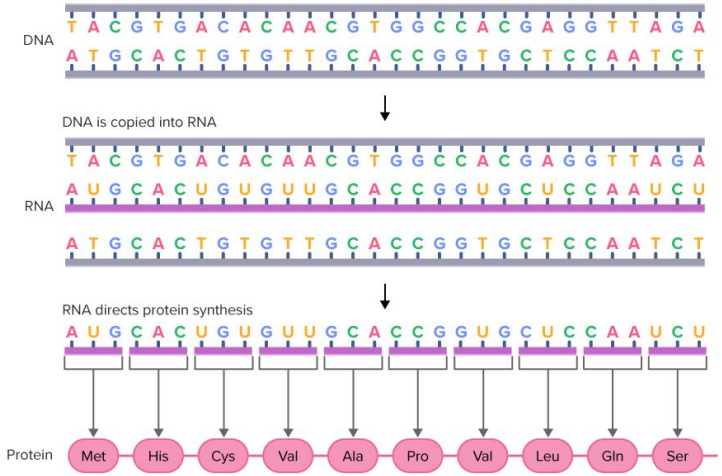
Department of Computer Science
College of Computer Studies
MSU - Iligan Institute of Technology

What is Sequence Data?

Data where the order matters. Prediction at any given time step is dependent on the previous steps.

Sequence Data Examples

- Text (Language Translation, Predicting next word)
- Time Series (Finance, Weather)
- Music and Speech
- DNA/Protein Sequences





Eric: Hi Tom! How are
Eric: Hi Tom! How are you?
Eric: Hi Tom! How are you? Tom:
Eric: Hi Tom! How are you? Tom: Sorry,
Eric: Hi Tom! How are you? Tom: Sorry, who
Eric: Hi Tom! How are you? Tom: Sorry, who are
Eric: Hi Tom! How are you? Tom: Sorry, who are you
Eric: Hi Tom! How are you? Tom: Sorry, who are you again?

Several iterations of an LLM predicting the next word of a sentence

Sequence Modeling Applications

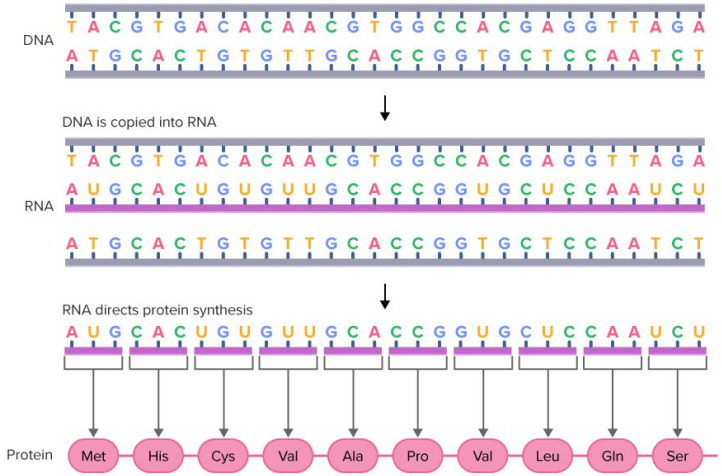
To model sequences effectively, networks must:

1. Handle **variable-length** sequences
2. Track **long-term dependencies**
3. Maintain order information or **sequence order**
4. **Share parameters** across time / sequence

Challenges of modeling sequences:

- Variable length inputs
- Long-term dependencies
- Order and context sensitivity





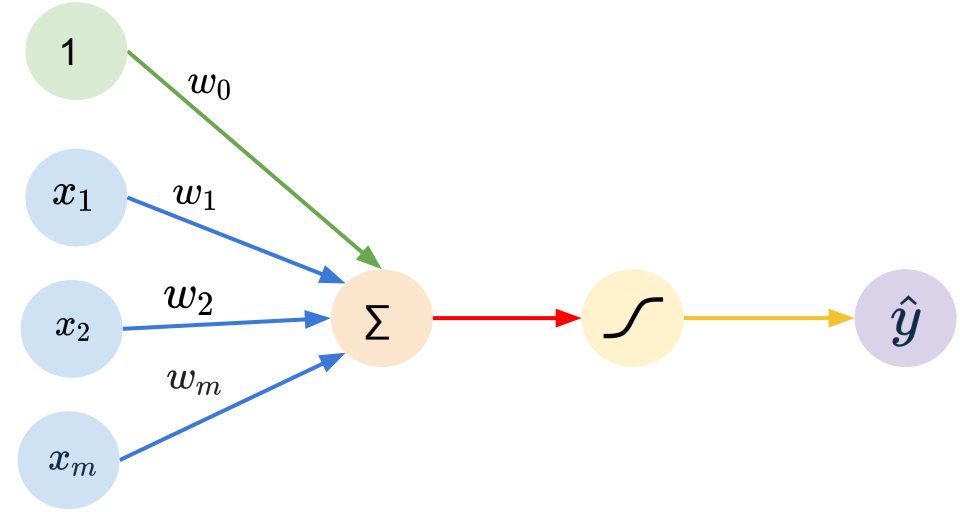
Eric: Hi Tom! How are
Eric: Hi Tom! How are you?
Eric: Hi Tom! How are you? Tom:
Eric: Hi Tom! How are you? Tom: Sorry,
Eric: Hi Tom! How are you? Tom: Sorry, who
Eric: Hi Tom! How are you? Tom: Sorry, who are
Eric: Hi Tom! How are you? Tom: Sorry, who are you
Eric: Hi Tom! How are you? Tom: Sorry, who are you again?

Several iterations of an LLM predicting the next word of a sentence

Building Neural Networks with Perceptrons

RECALL

Perceptron: Simplified

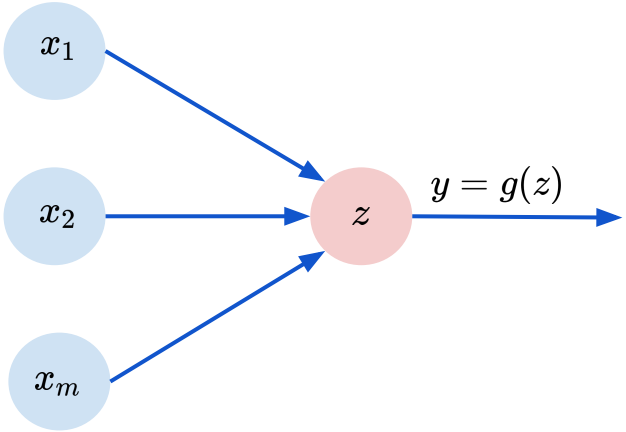


$$\hat{y} = g(\, w_0 + X^T \, W)$$

where $X = \begin{bmatrix} x_1 \\ \cdot \\ \cdot \\ x_m \end{bmatrix}$ and $W = \begin{bmatrix} w_1 \\ \cdot \\ \cdot \\ w_m \end{bmatrix}$

RECALL

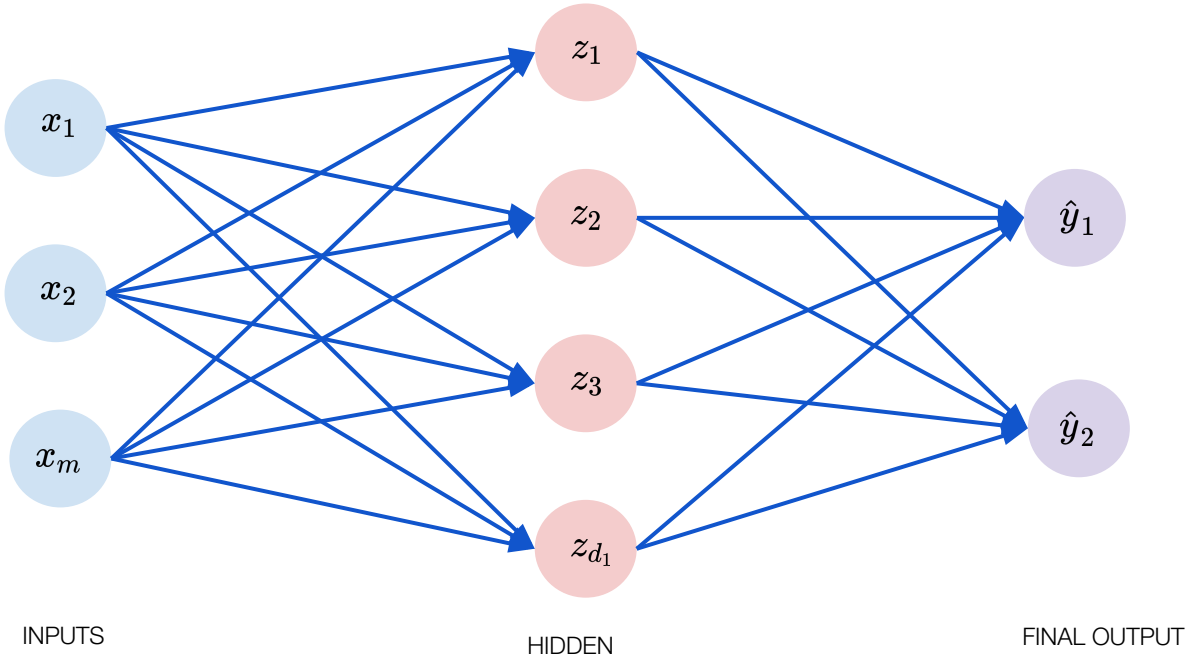
Perceptron: Simplified



$$z = w_0 + \sum_{j=1}^{m} x_j \;\; w_j$$

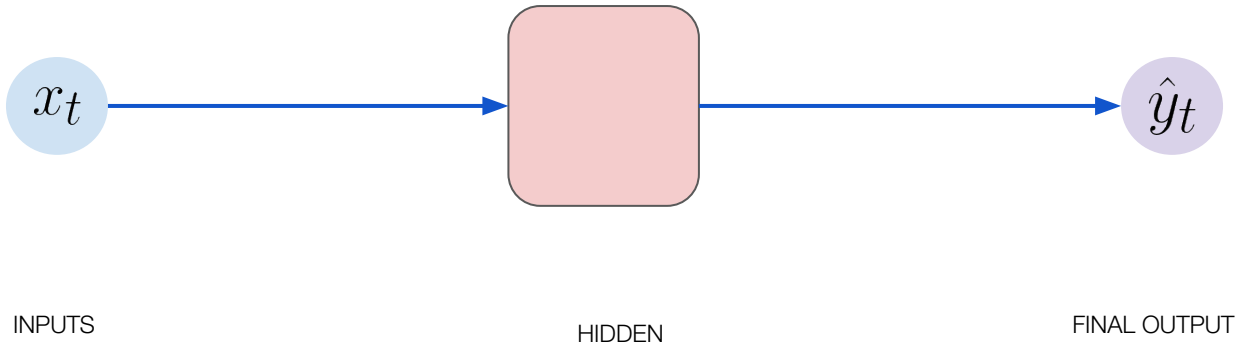
RECALL

Feed Forward Neural Network

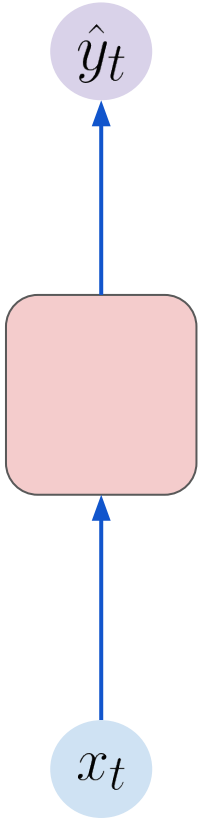


RECALL

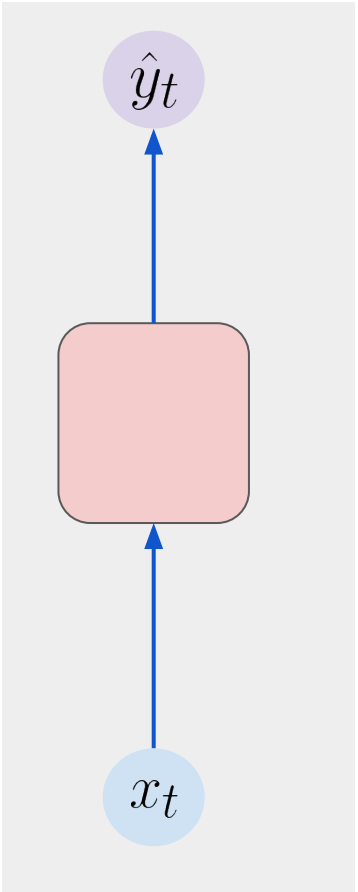
Feed Forward Neural Network



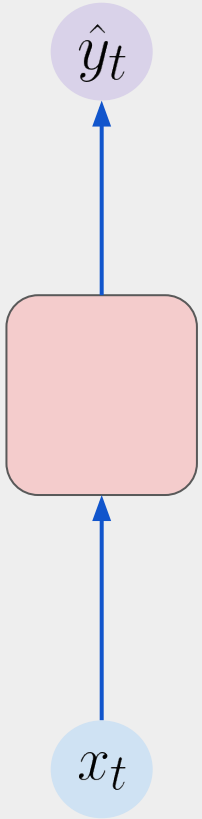
Individual Timesteps

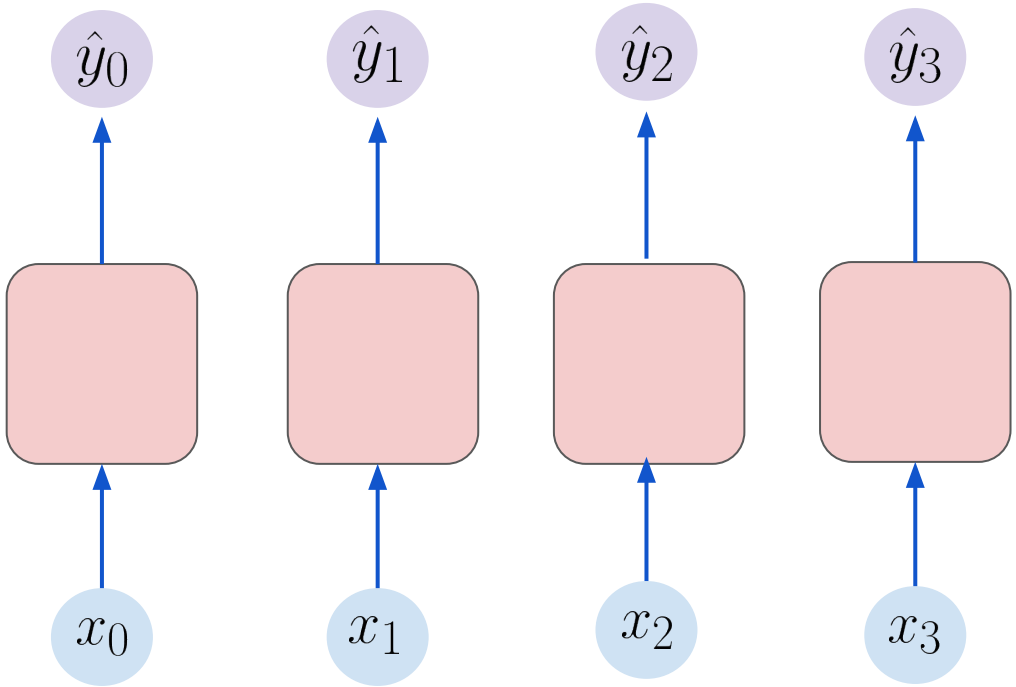


Individual Timesteps



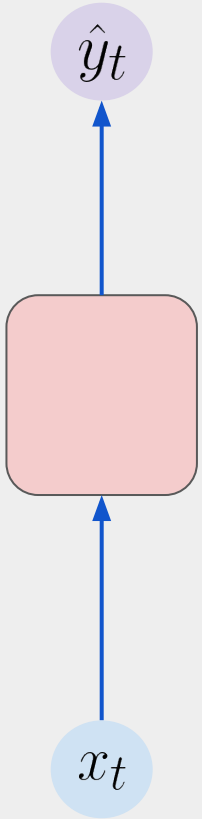
Individual Timesteps

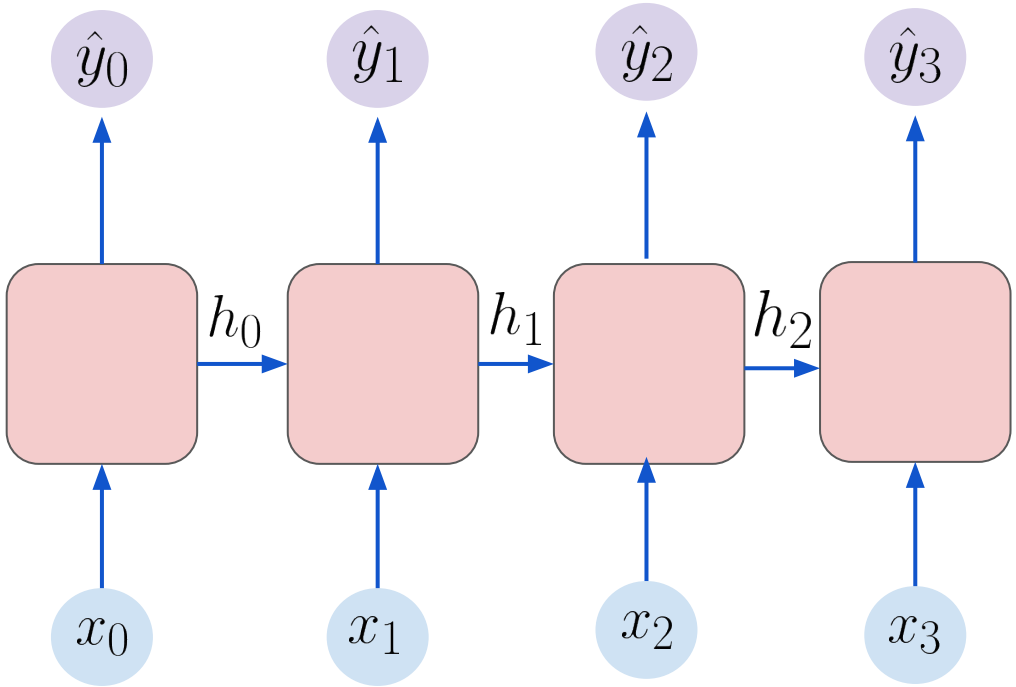




$$\hat{y}_t = f(x_t)$$

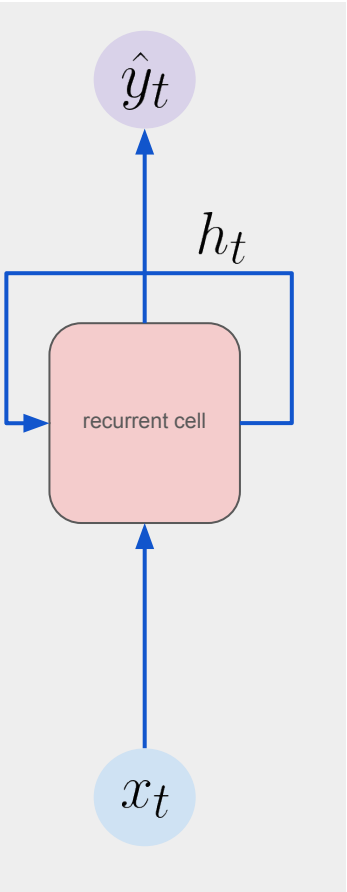
Individual Timesteps

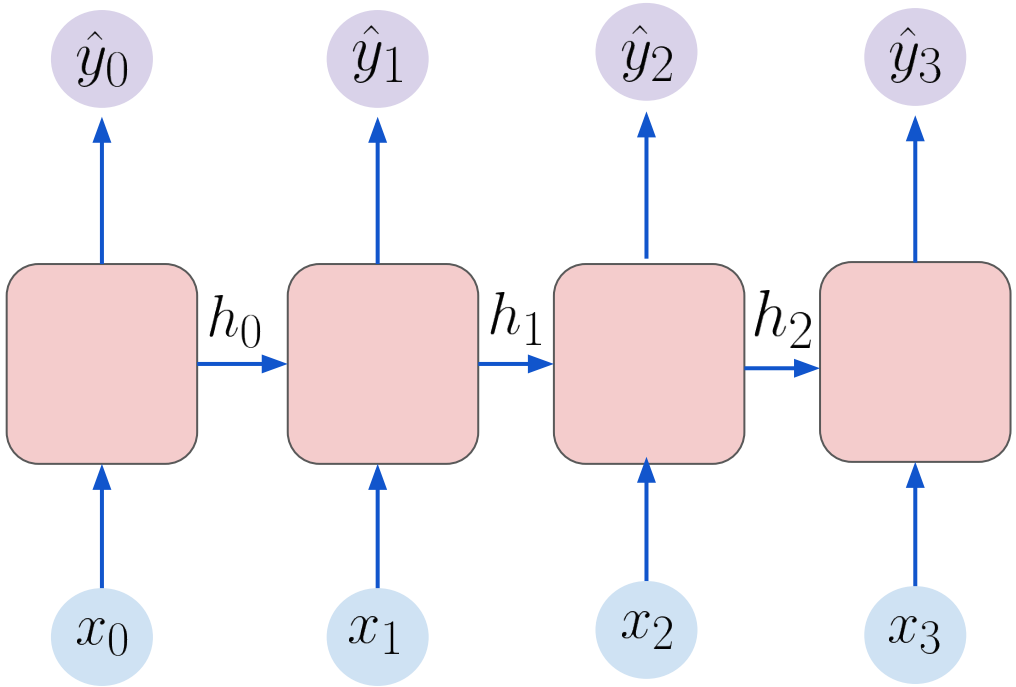




$$\hat{y}_t = f(x_t, h_{t-1})$$

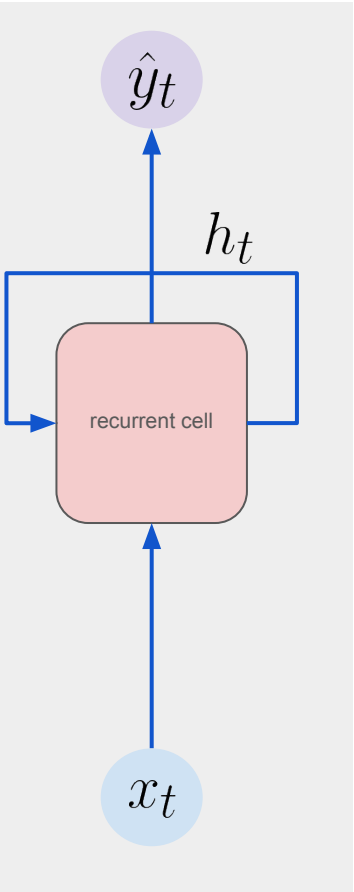
Individual Timesteps

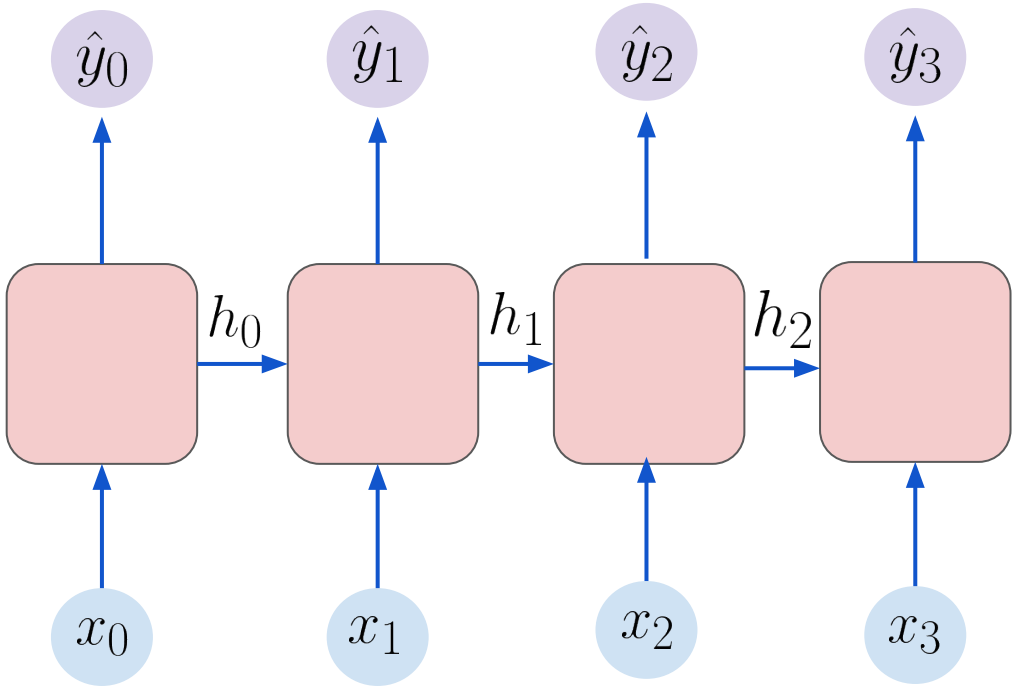




$$\hat{y}_t = f(x_t, h_{t-1})$$

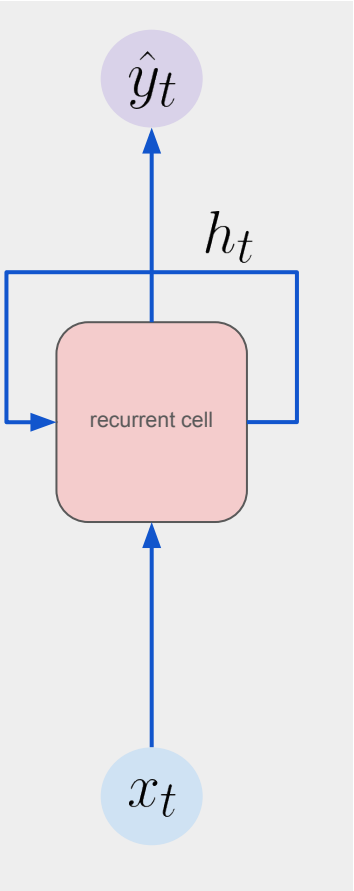
Recurrent Neural Network (RNN)

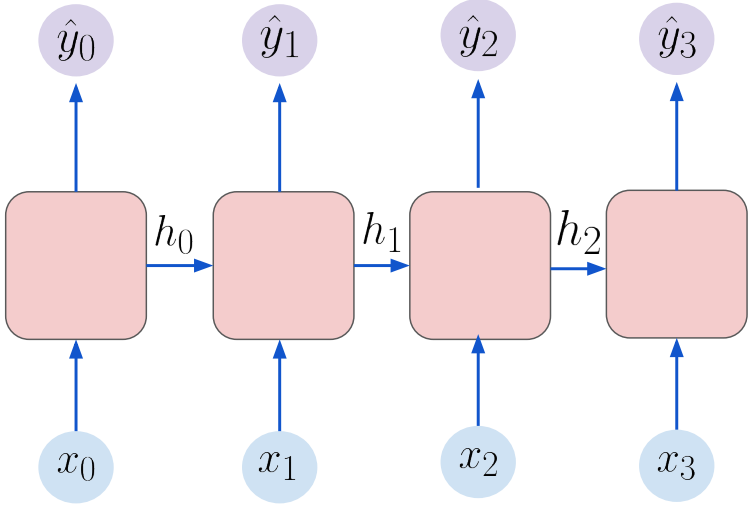




$$\hat{y}_t = f(x_t, h_{t-1})$$

Recurrent Neural Network (RNN)

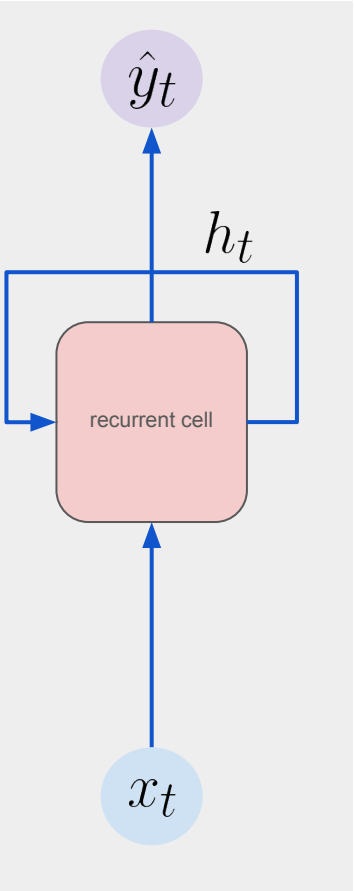


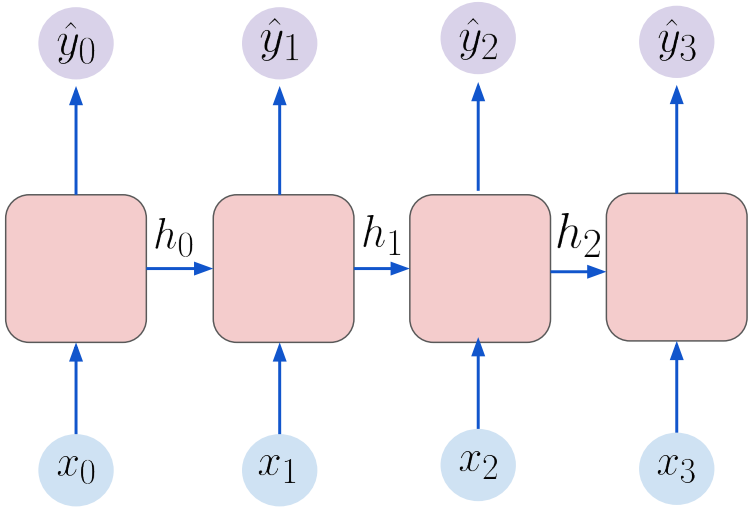


$$\hat{y}_t = f(x_t, h_{t-1})$$

The output of each time step depends on previous hidden states — memory through recurrence.

Recurrent Neural Network (RNN)





$$\hat{y}_t = f(x_t, h_{t-1})$$

At each time step t , for every input x_t and hidden state h_{t-1}

output vector

$$\hat{y}_t = W_{hy} h_t$$

update hidden state

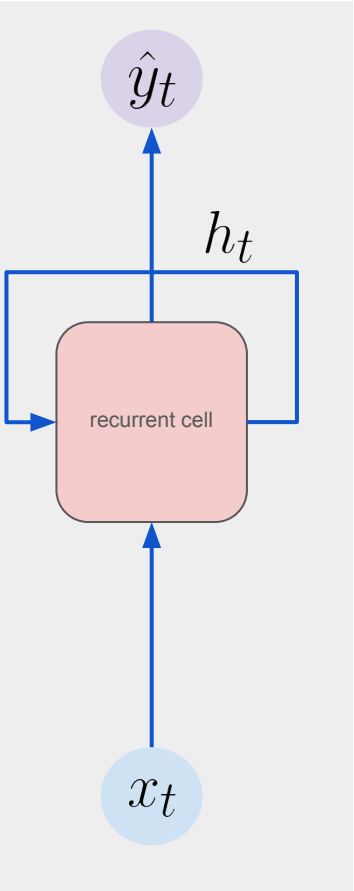
$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$$

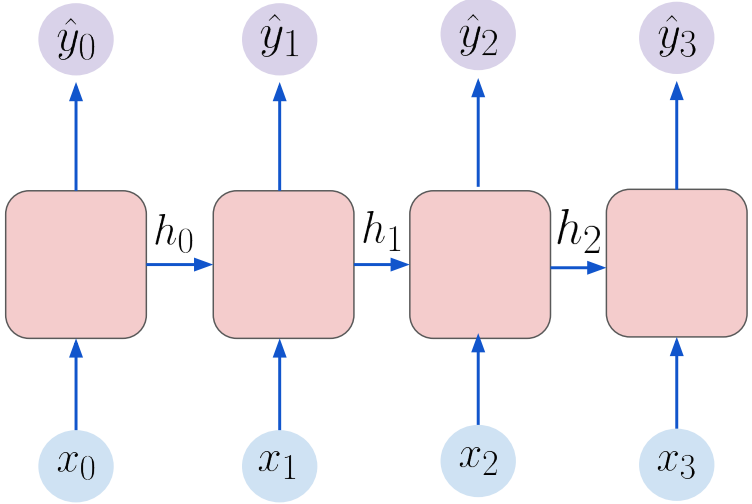
input vector

$$x_t$$

The output of each time step depends on previous hidden states — memory through recurrence.

Recurrent Neural Network (RNN)





$$\hat{y}_t = f(x_t, h_{t-1})$$

At each time step t , for every input x_t and hidden state h_{t-1}

output vector

$$\hat{y}_t = W_{hy} h_t$$

update hidden state

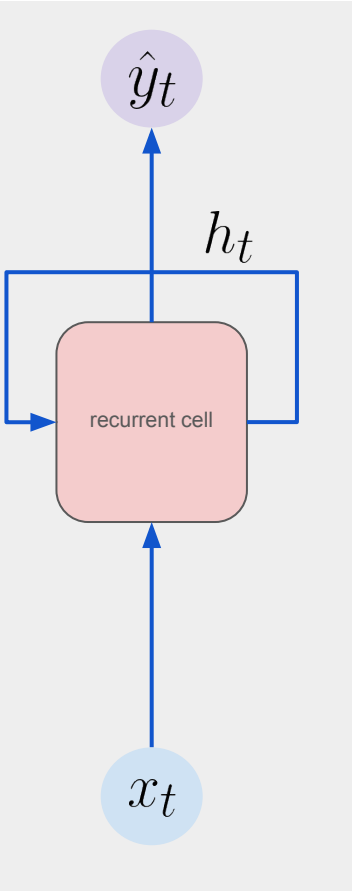
$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$$

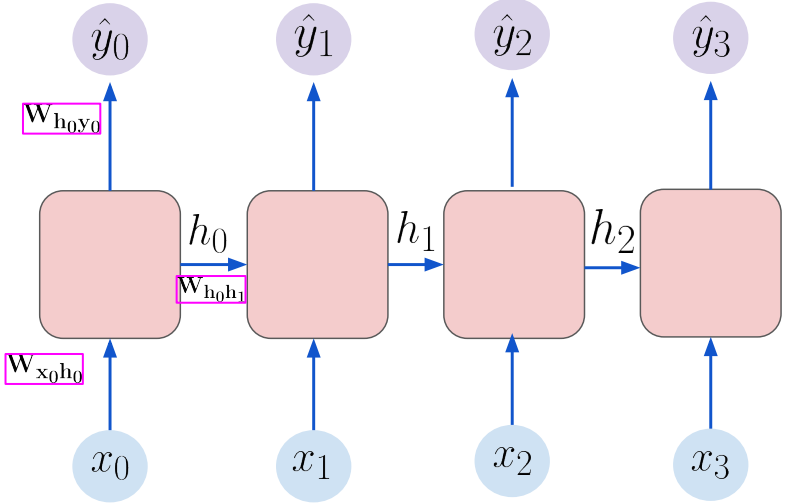
input vector

$$x_t$$

The same weights are reused at every step - parameter sharing across all the time steps

Recurrent Neural Network (RNN)





$$\hat{y}_t = f(x_t, h_{t-1})$$

W_{xh} INPUT TO HIDDEN WEIGHT MATRIX

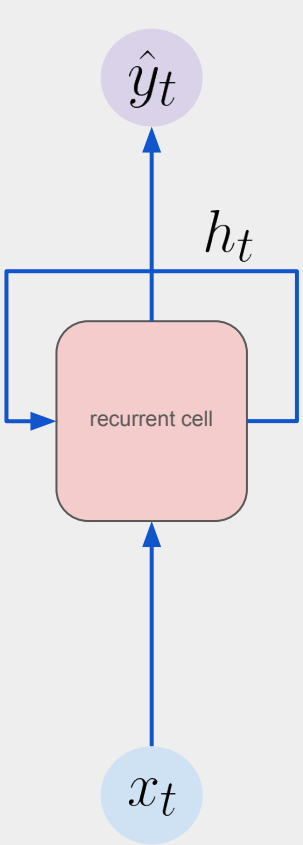
W_{hh} HIDDEN TO HIDDEN WEIGHT MATRIX

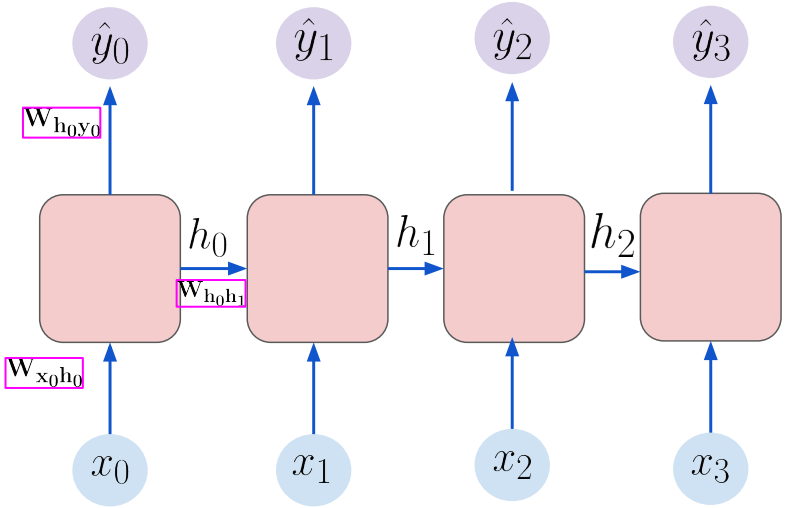
W_{hy} HIDDEN TO OUTPUT WEIGHT MATRIX

The same weights are reused at every step - parameter sharing across all the time steps

Recurrent Neural Network (RNN)

At each time step t , for every input x_t and hidden state h_{t-1}





$$\hat{y}_t = f(x_t, h_{t-1})$$

output vector

$$\hat{y}_t = W_{hy} h_t$$

update hidden state

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$$

input vector

x_t

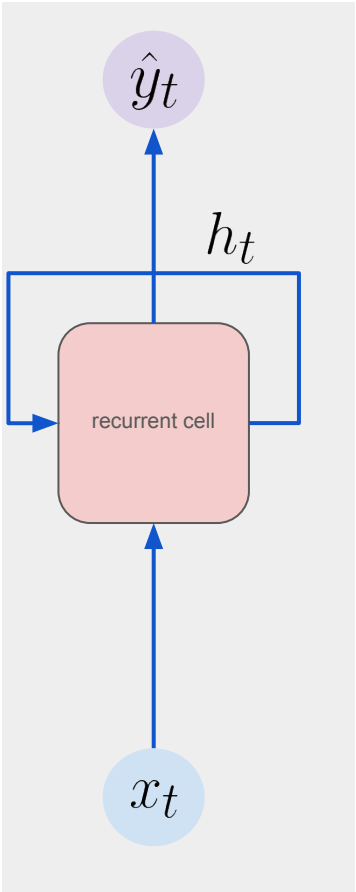
W_{xh} INPUT TO HIDDEN WEIGHT MATRIX

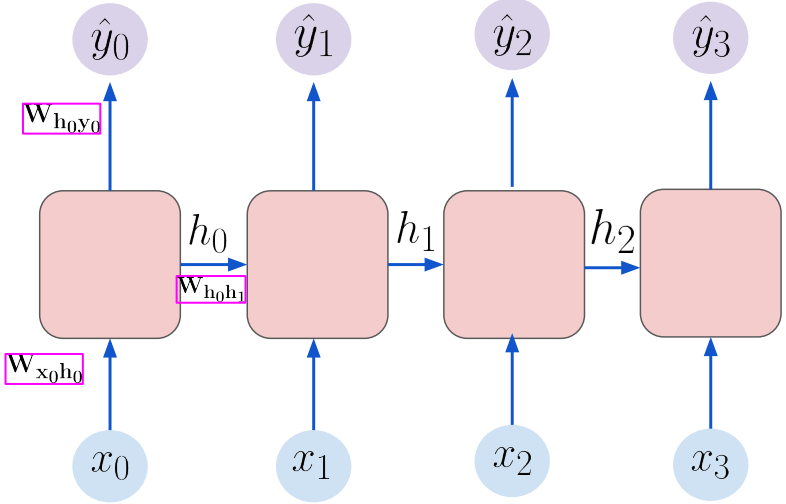
W_{hh} HIDDEN TO HIDDEN WEIGHT MATRIX

W_{hy} HIDDEN TO OUTPUT WEIGHT MATRIX

The same weights are reused at every step - parameter sharing across all the time steps

Recurrent Neural Network (RNN)





$$\hat{y}_t = f(x_t, h_{t-1})$$

At each time step t , for every input x_t and hidden state h_{t-1}

output vector

$$\hat{y}_t = W_{hy} h_t$$

update hidden state

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$$

Keep state values stable since its squashes its input range [-1,1]

input vector

x_t

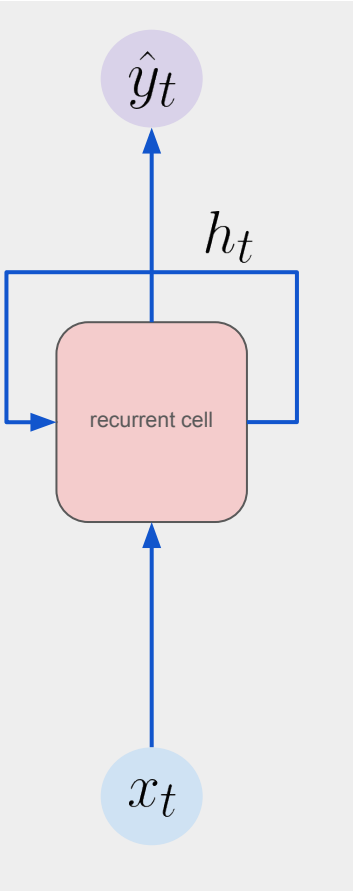
W_{xh} INPUT TO HIDDEN WEIGHT MATRIX

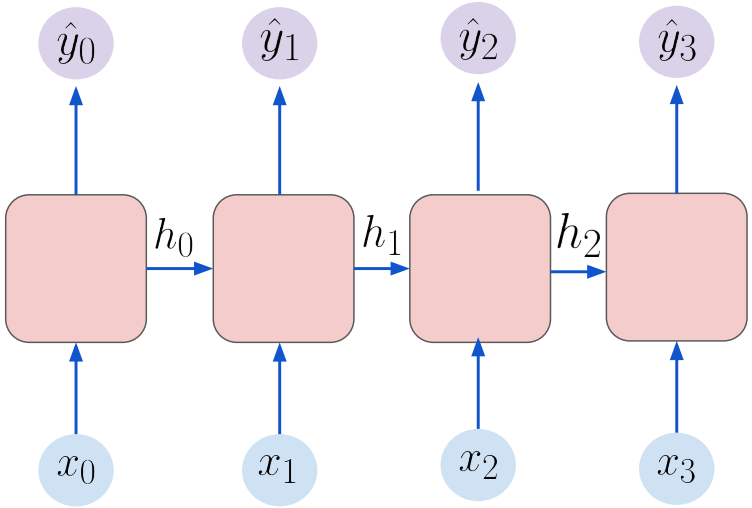
W_{hh} HIDDEN TO HIDDEN WEIGHT MATRIX

W_{hy} HIDDEN TO OUTPUT WEIGHT MATRIX

The same weights are reused at every step - parameter sharing across all the time steps

Recurrent Neural Network (RNN)





$$\hat{y}_t = f(x_t, h_{t-1})$$

At each time step t , for every input x_t and hidden state h_{t-1}

output vector

$$\hat{y}_t = W_{hy} h_t \quad \hat{y}_t = W_{hy}^T h_t$$

update hidden state

$$h_t = \tanh(W_{hh}^T h_{t-1} + W_{xh}^T x_t)$$

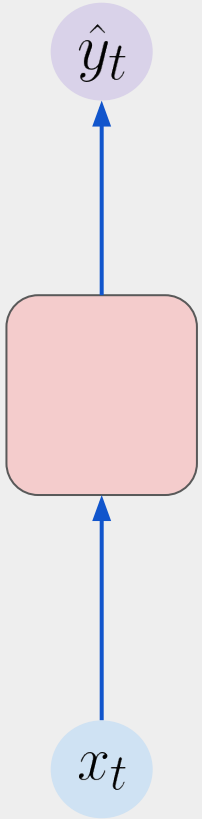
$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$$

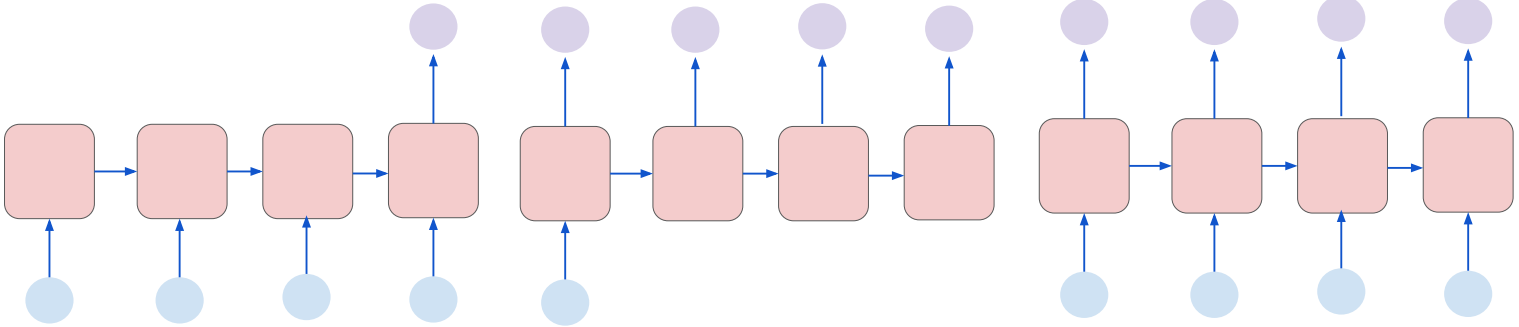
input vector

$$x_t$$

The output of each time step depends on previous hidden states — memory through recurrence.

RNN for Sequence Modeling



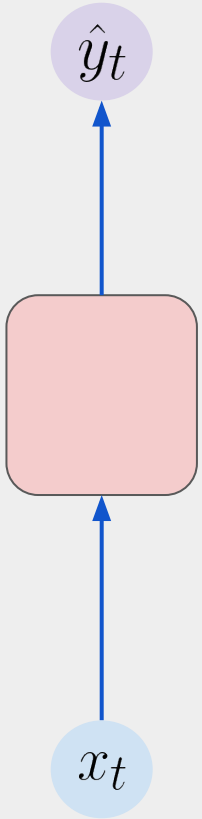


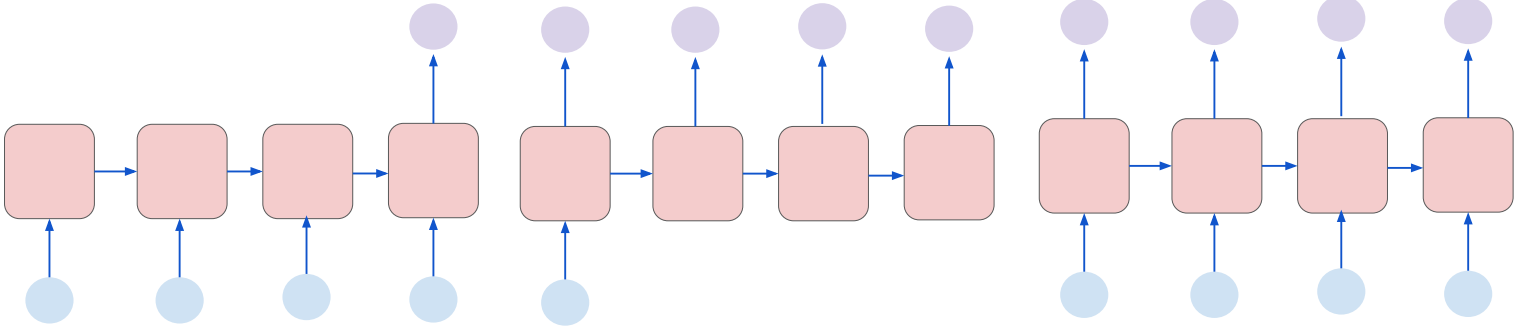
Many to One
- Sentiment Classification

One to Many
- Text Generation
- Image Captioning

Many to Many
- Speech Translation
- Forecasting

RNN for Sequence Modeling





Many to One
- Sentiment Classification

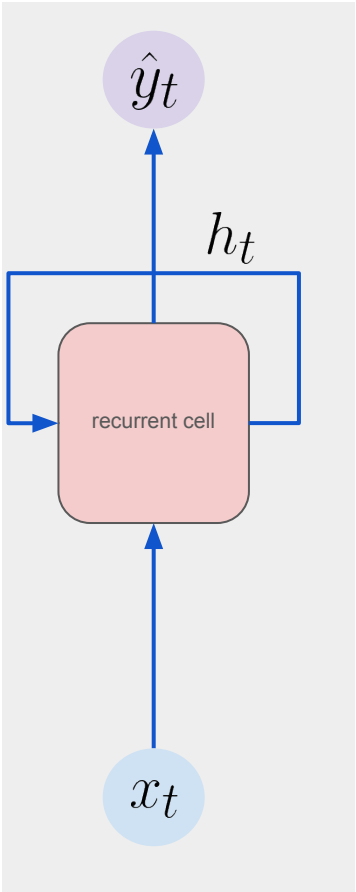
One to Many
- Text Generation
- Image Captioning

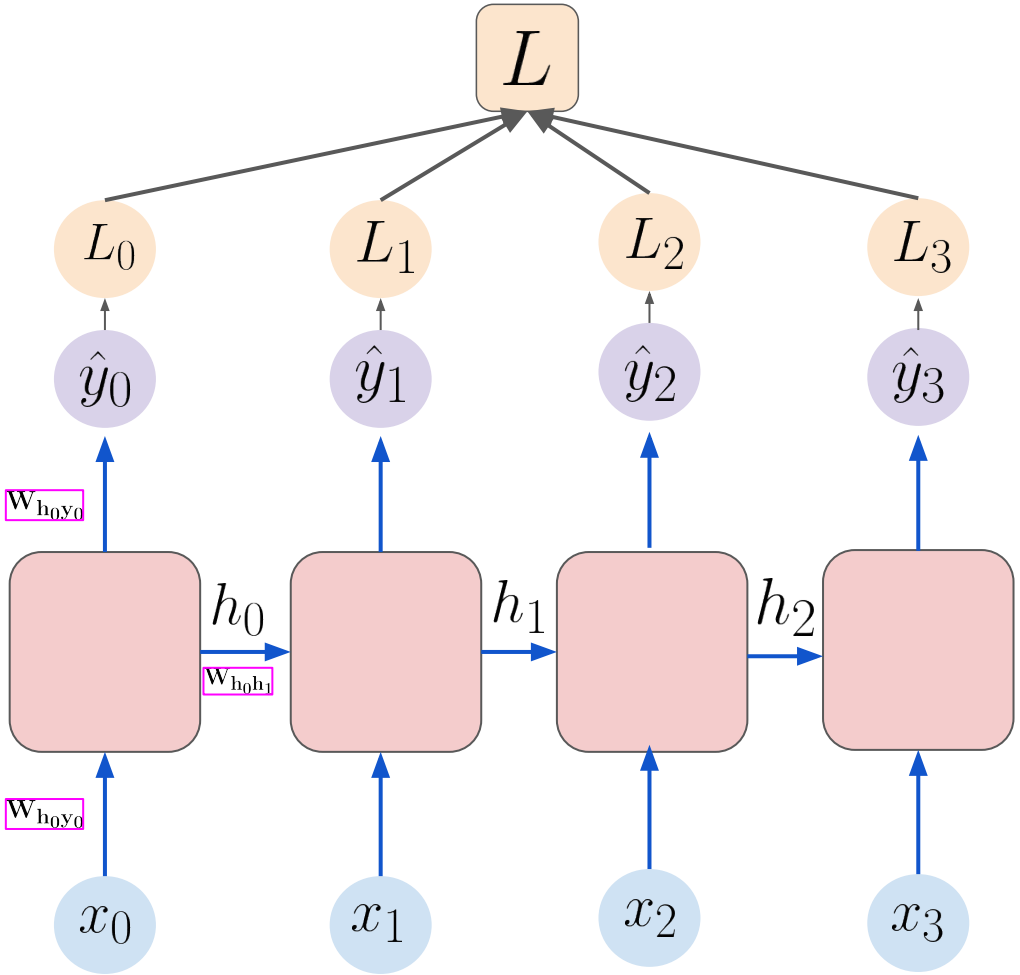
Many to Many
- Speech Translation
- Forecasting

Key Design Criteria:

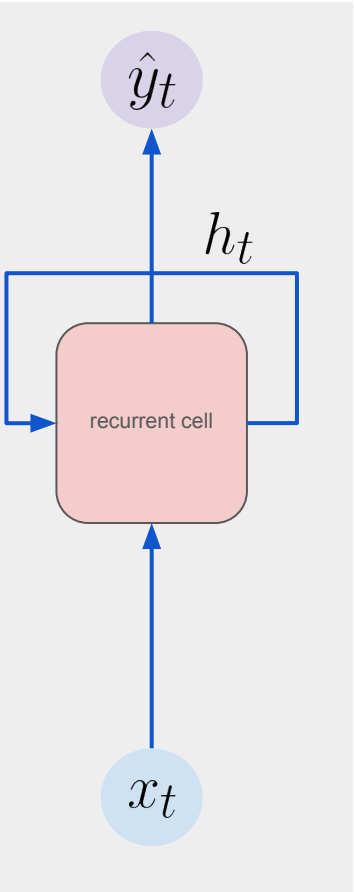
1. Handle variable-length sequences
2. Track long-term dependencies
3. Maintain information about order
4. Share parameters across the sequence

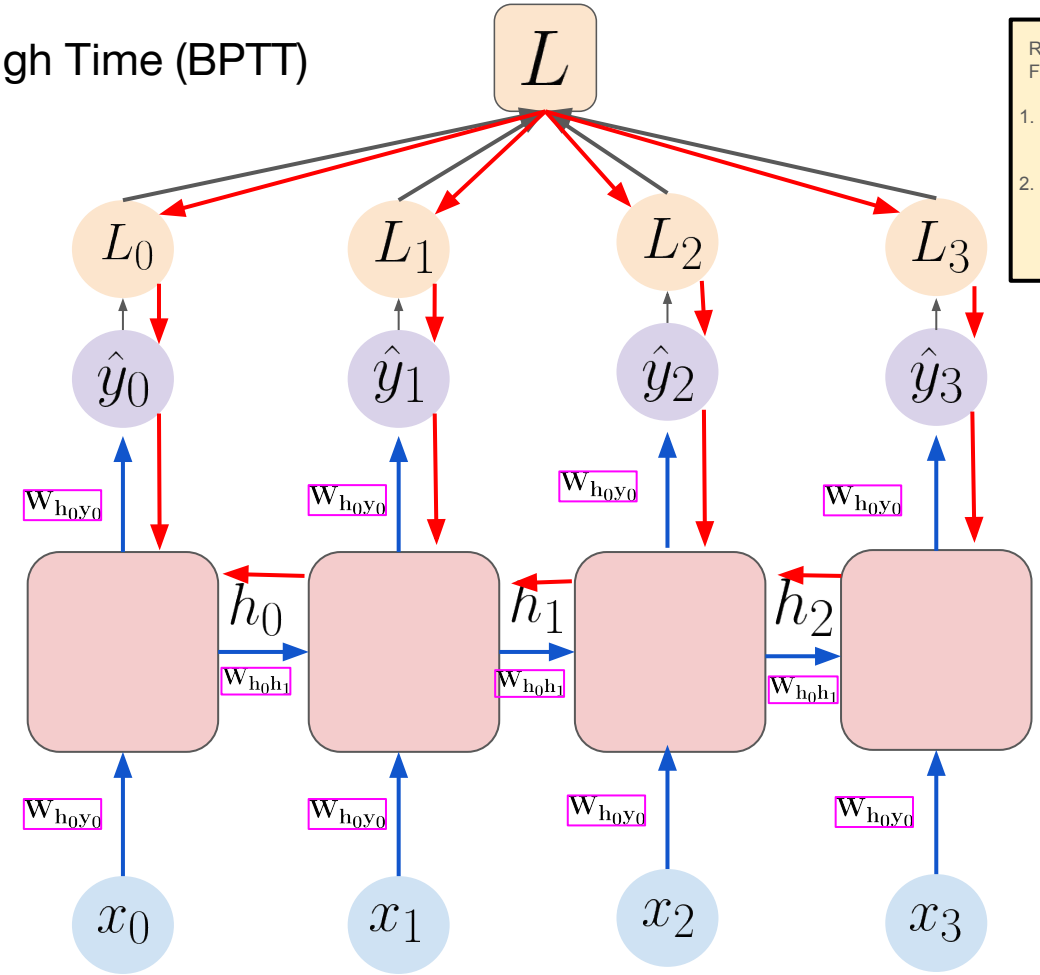
Forward Propagation





Backward Propagation Through Time (BPTT)



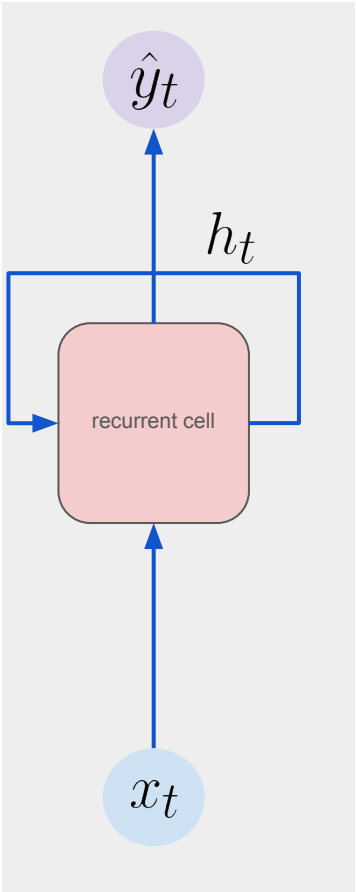


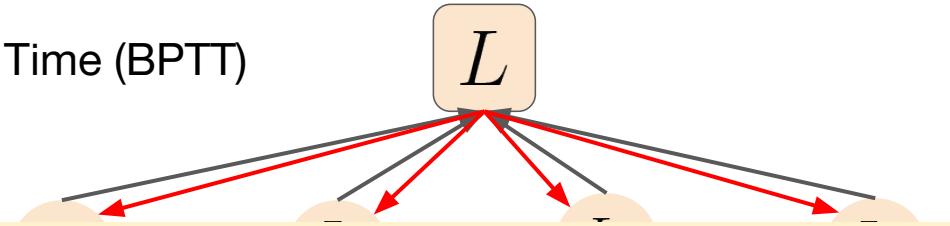
Recall Backpropagation in Feed Forward Network

1. Take the derivative (gradient) of the loss with respect to each parameter
2. Adjust weights to shift parameters in order to minimize loss

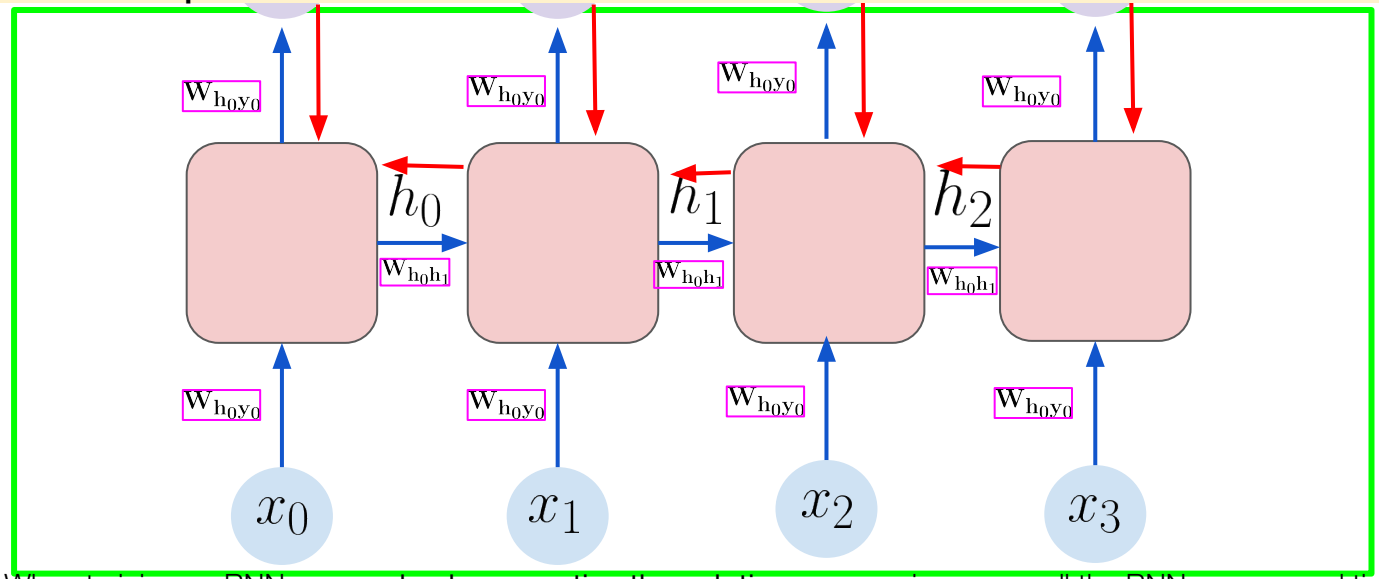
When training an RNN, we use **backpropagation through time** — meaning we unroll the RNN over several time steps and **propagate the error backwards through all those steps.**

Backward Propagation Through Time (BPTT)





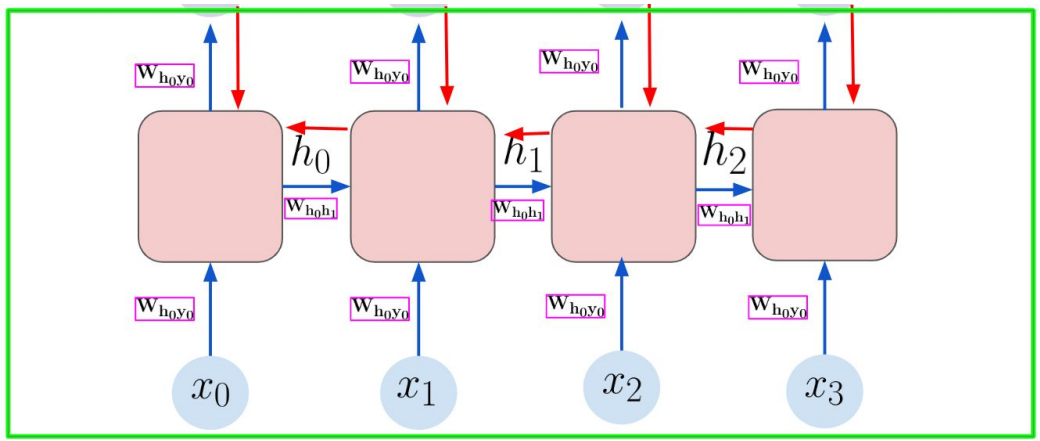
Computing the gradient with respect to h_0 involves many factors of W_{hh} due to repeated gradient computation across time steps



When training an RNN, we use **backpropagation through time** — meaning we unroll the RNN over several time steps and **propagate the error backwards through all those steps.**

When computing gradients backward through time (BPTT), we repeatedly apply the chain rule

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

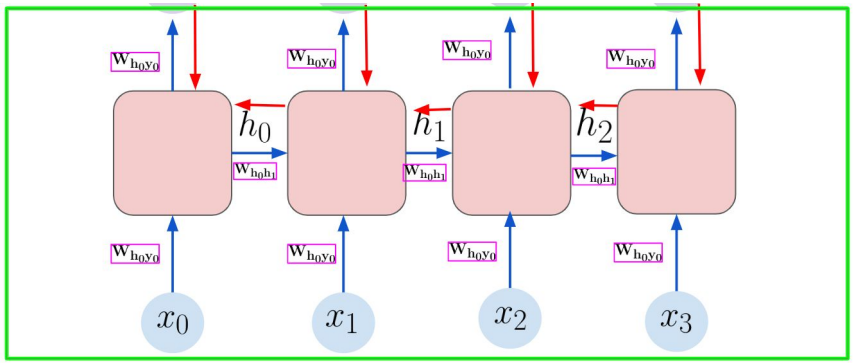


Where each term can be approximated as,

$$\frac{\partial h_k}{\partial h_{k-1}} \approx W_{hh} \cdot \mathrm{diag}(f'(a_k))$$

The gradient back to an early timestep involves multiplying many matrices together wherein each multiplication either **amplifies** or **shrinks** the gradient depending on the magnitude of these matrices

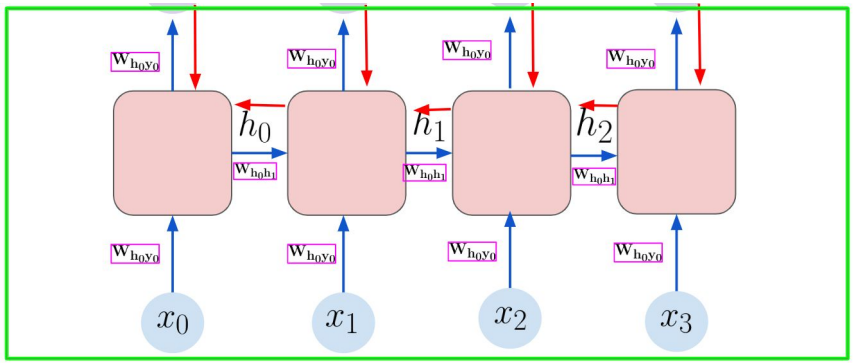
$$\frac{\partial L}{\partial h_0} \propto \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_t)\right)\right)^T \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_{t-1})\right)\right)^{T-1} \cdots \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_1)\right)\right)$$



$$\frac{\partial L}{\partial h_0} \propto \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_t)\right)\right)^T \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_{t-1})\right)\right)^{T-1} \cdots \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_1)\right)\right)$$

If we look at the largest eigenvalue magnitude of $W_{hh} \cdot \mathrm{diag}(f'(a_k))$

Eigenvalue	Effect on Gradients	Problem
< 1	Gradients shrink exponentially	Vanishing gradient
> 1	Gradients blow up exponentially	Exploding gradient



$$\frac{\partial L}{\partial h_0} \propto \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_t)\right)\right)^T \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_{t-1})\right)\right)^{T-1} \cdots \left(W_{hh} \cdot \mathrm{diag}\left(f'(a_1)\right)\right)$$

If we look at the largest eigenvalue magnitude of $W_{hh} \cdot \mathrm{diag}(f'(a_k))$

Eigenvalue	Effect on Gradients	Problem
< 1	Gradients shrink exponentially	Vanishing gradient
> 1	Gradients blow up exponentially	Exploding gradient

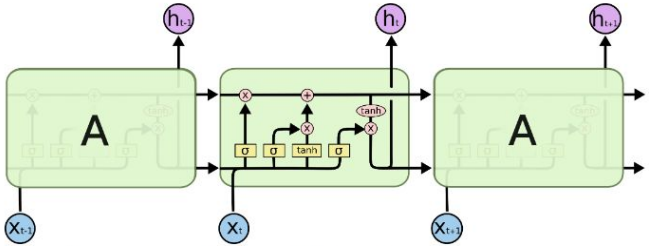
.. challenge and limitation of RNN

Gated Models: Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU)

- Are advanced RNN architectures designed to solve the limitations found in classic RNN using gates that regulate information that flow through the network
- Mitigate vanishing and exploding gradient problems by controlling what to remember or forget
- Still limited by sequential computation, since each timestep depends on the previous one

Long Short-Term Memory (LSTM)

- Memory system is implemented using gates:
  - **forget** (what information from previous cell state to discard)
  - **input** (what new information from the current input should be stored in the cell state),
  - **output** (what part of the current cell state is used to compute the next hidden state which is passed to the next time step)
- Maintains a hidden state (long-term memory) and cell state (short-term memory)



The repeating module in an LSTM contains four interacting layers.

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Gated Models: Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU)

- Are advanced RNN architectures designed to solve the limitations found in classic RNN using gates that regulate information that flow through the network
- Mitigate vanishing and exploding gradient problems by controlling what to remember or forget
- Still limited by sequential computation, since each timestep depends on the previous one

Gated Recurrent Unit (GRU)

- Combine the input and forget gates of LSTM into a single update gate and reset gate
- Maintains a single hidden state

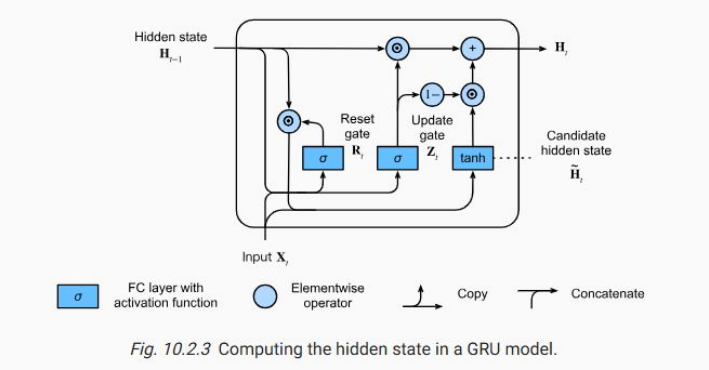


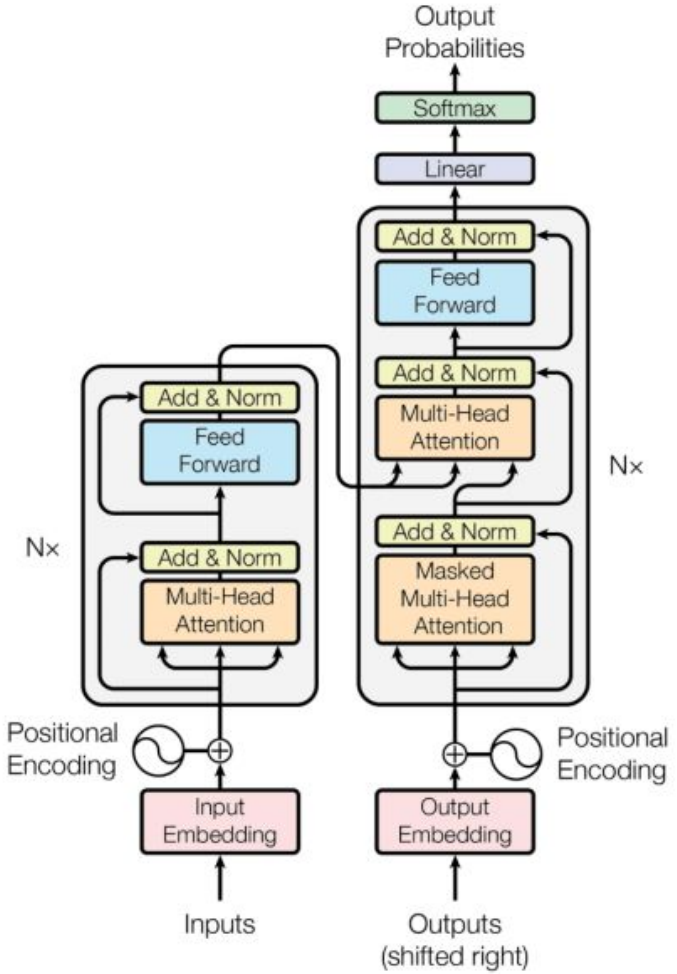
Fig. 10.2.3 Computing the hidden state in a GRU model.

https://d2l.ai/chapter_recurrent-modern/gru.html

Eliminating Recurrence: Attention is All You Need

In 2017, Vaswani et al. introduced a breakthrough paper titled "Attention Is All You Need", proposing the **Transformer architecture**, which completely eliminated recurrence and relied solely on **attention mechanisms** to draw global dependencies between input and output.

By moving away from the sequential nature of RNNs, Transformers can process entire sequences in parallel using a novel concept known as **self-attention**.



Input in English:
“The cat sat on the mat”

Output in Tagalog:
“Ang pusa ay umupo sa banig”

A transformer model uses **attention** to decide which English words are most relevant when generating each Tagalog word.

Attention is a mechanism that models dependencies between different sequences when generating an output.

Self-Attention is a mechanism that models dependencies within the same sequence, allowing each position to relate to others and build contextual representations.

“The cat sat on the mat”

Word Being Processed	Other Words It Attends To	Why
sat	**cat**	To understand *who* sat
	mat	To understand *where* it sat
	The, on, the	Less important context

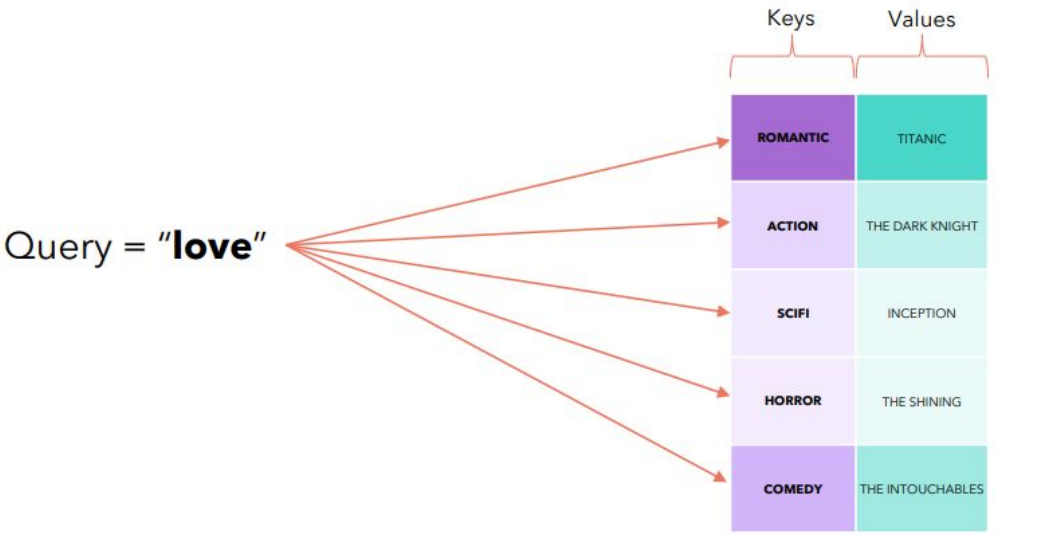
A transformer model uses **self-attention** to understand the relationship between words within the same sentence.

Model assigns attention weights -- higher values to more relevant words and lower values to less relevant ones.

Attention is a mechanism that models dependencies between different sequences when generating an output.

Self-Attention is a mechanism that models dependencies within the same sequence, allowing each position to relate to others and build contextual representations.

The self-attention mechanism allows a model to understand the relationships between words in a sentence, regardless of their distance from each other. It works by computing a score that determines how much focus each word should have on every other word in the sentence.

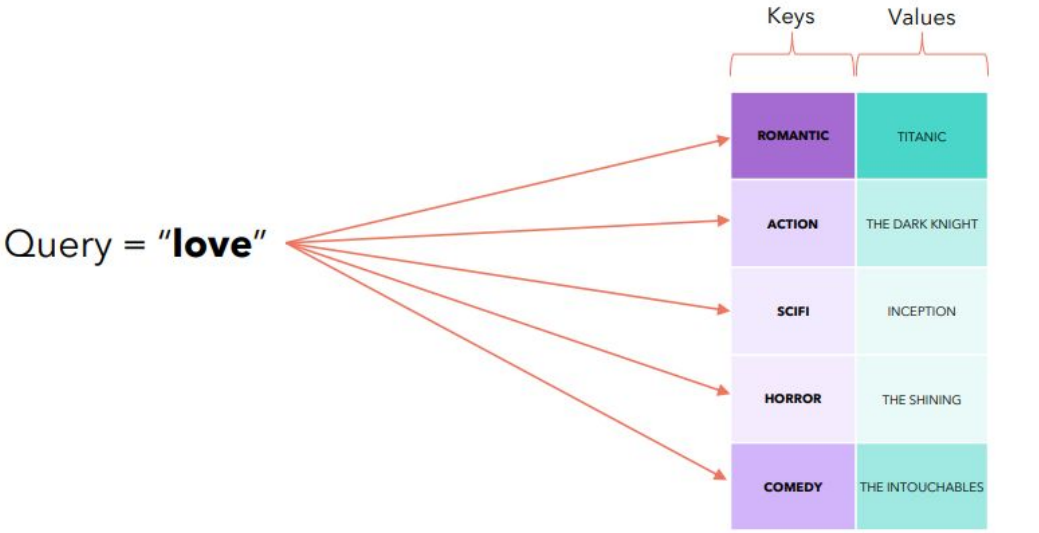


* this could be a Python dictionary or a database table.

Attention is a mechanism that models dependencies between different sequences when generating an output.

Self-Attention is a mechanism that models dependencies within the same sequence, allowing each position to relate to others and build contextual representations.

The self-attention mechanism allows a model to understand the relationships between words in a sentence, regardless of their distance from each other. It works by computing a score that determines how much focus each word should have on every other word in the sentence.



* this could be a Python dictionary or a database table.

The key mathematical building blocks of self-attention are three learned vectors derived from each input token:

1. Query (Q) - what the token is looking for in other tokens
2. Key (K) - properties of other tokens to be compared with the query
3. Value (V) - information that will be passed along based on how much attention the query places on each key

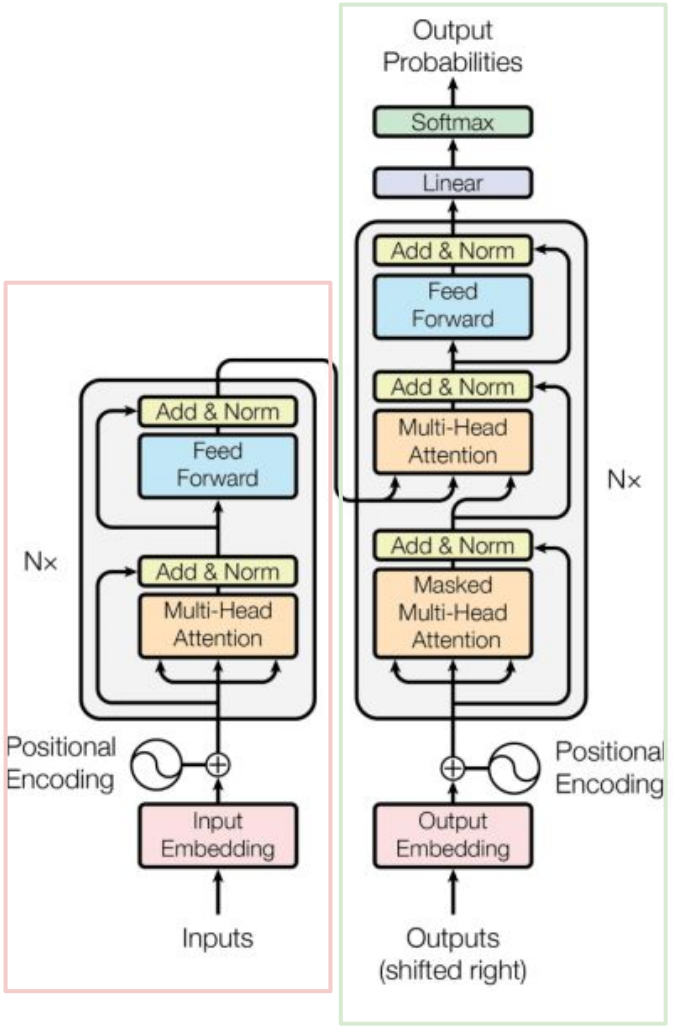
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Eliminating Recurrence: Attention is All You Need

The Transformer model consists of two main parts:
the encoder and the decoder which work together for sequence-to-sequence tasks.

The encoder processes the input sequence and captures its most important features, while the decoder uses this information to generate the output sequence.

Together, they enable the model to identify and attend to the most relevant parts of the input when producing an accurate output.



Transformer Architecture: Encoder

The encoder takes in a sequence and processes it in several layers. Each layer consists of two main components: **multi-head self-attention** and a **feed-forward neural network**.

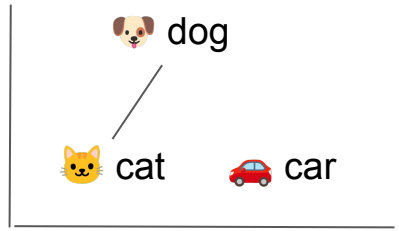
Input Embedding

Each word in the input sequence is first mapped to a unique position in a predefined **vocabulary** — a collection of all words the model has learned during training.

Every word is assigned a **unique ID or index**, which is then converted into a dense numerical vector called an **embedding**.

These embeddings represent the **semantic meaning of words in a high-dimensional space**. In this space, each word corresponds to a unique vector representation, allowing the model to measure similarity and relationships between words based on distance and direction in vector space.





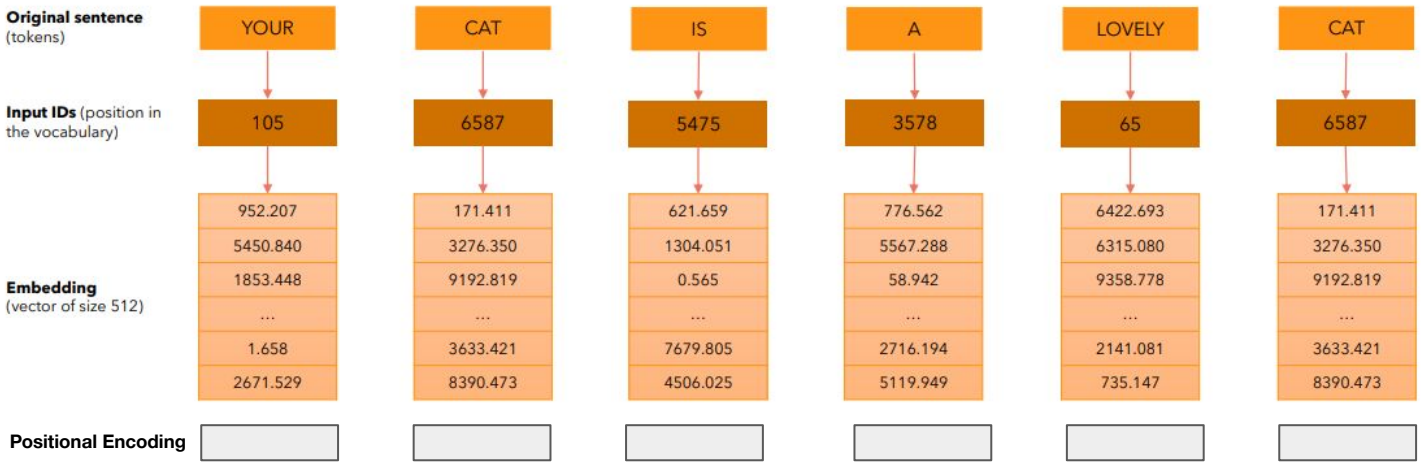
Transformer Architecture: Encoder

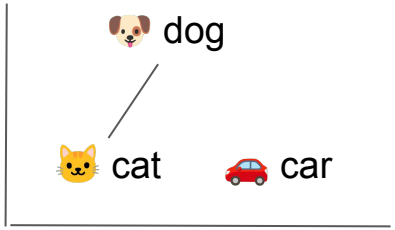
The encoder takes in a sequence and processes it in several layers. Each layer consists of two main components: **multi-head self-attention** and a **feed-forward neural network**.

Positional Encoding

Transformers process all words in a sequence simultaneously. To preserve the position and order of words, the model adds a positional encoding to each word embedding using sine and cosine functions.

Sine and cosine functions create smooth, repeating numerical patterns that encode **word positions** that give the Transformer a built-in sense of **sequence order**, even though it processes all words **in parallel**.





Transformer Architecture: Encoder

The encoder takes in a sequence and processes it in several layers. Each layer consists of two main components: **multi-head self-attention** and a **feed-forward neural network**.

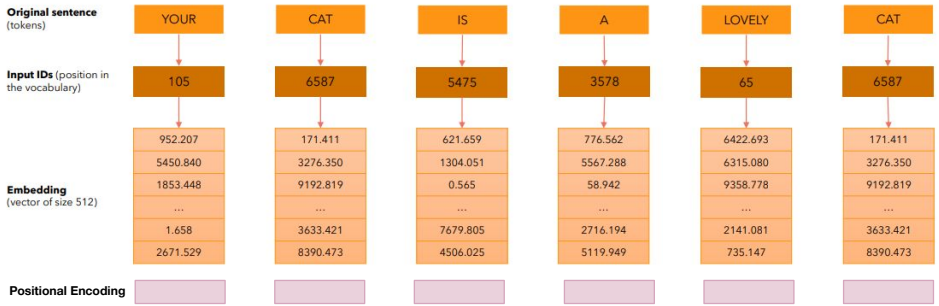
Positional Encoding

Transformers process all words in a sequence simultaneously. To preserve the position and order of words, the model adds a positional encoding to each word embedding using sine and cosine functions.

Each position (word order) gets a unique pattern of sine and cosine values that form unique “signatures” for every position

- Lower dimensions use slower wave patterns (long wavelengths)
- Higher dimensions use faster oscillations (short wavelengths)

Sine and cosine are continuous and periodic, the model can easily learn relative positions and reconstruct word and order relationships.

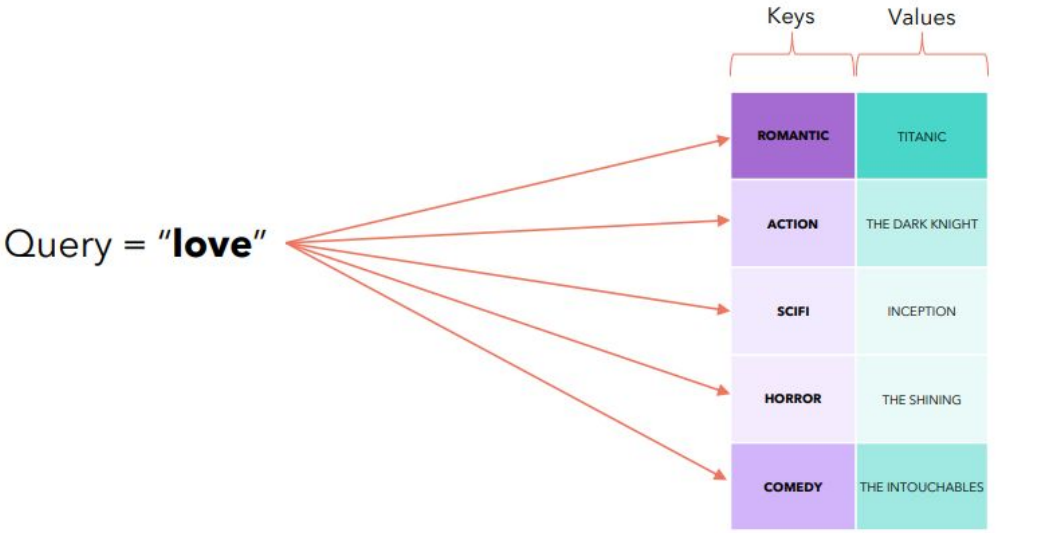


$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$\text{Input to Transformer} = \text{Word Embedding} + \text{Positional Encoding}$$

The self-attention mechanism allows a model to understand the relationships between words in a sentence, regardless of their distance from each other. It works by computing a score that determines how much focus each word should have on every other word in the sentence.



* this could be a Python dictionary or a database table.

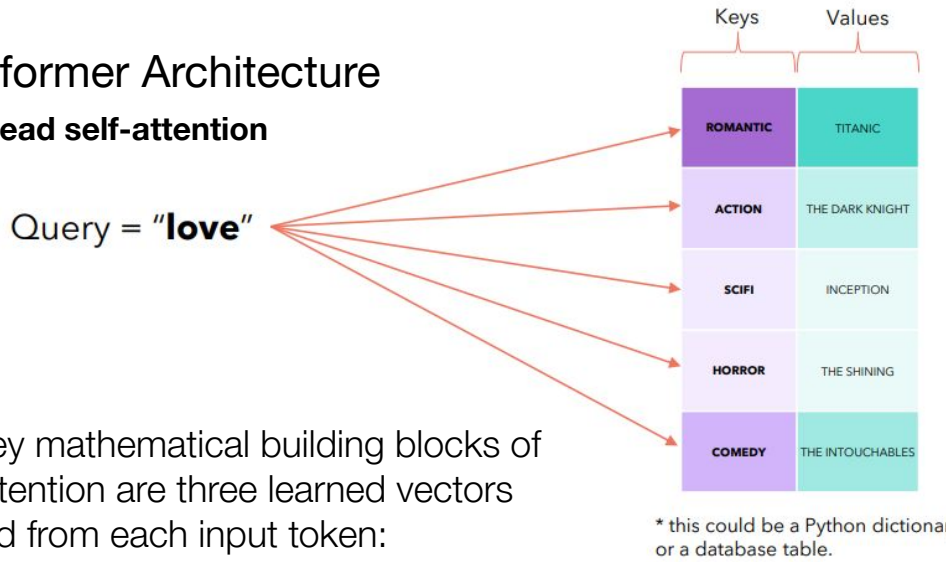
The key mathematical building blocks of self-attention are three learned vectors derived from each input token:

1. Query (Q) - what the token is looking for in other tokens
2. Key (K) - properties of other tokens to be compared with the query
3. Value (V) - information that will be passed along based on how much attention the query places on each key

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Transformer Architecture

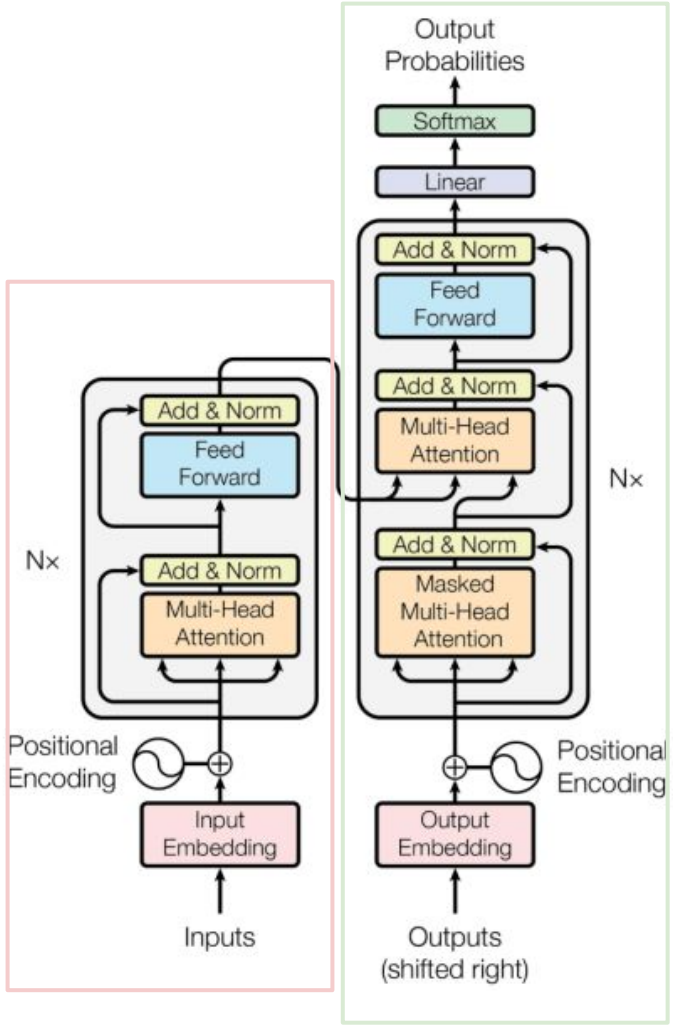
Multi-head self-attention



The key mathematical building blocks of self-attention are three learned vectors derived from each input token:

1. **Query (Q)** - what the token is looking for in other tokens
2. **Key (K)** - properties of other tokens to be compared with the query
3. **Value (V)** - information that will be passed along based on how much attention the query places on each key

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{softmax}\left(\frac{Q}{(6, 512)} \times \frac{K^T}{(512, 6)}\right) =$$

$$\sqrt{512}$$

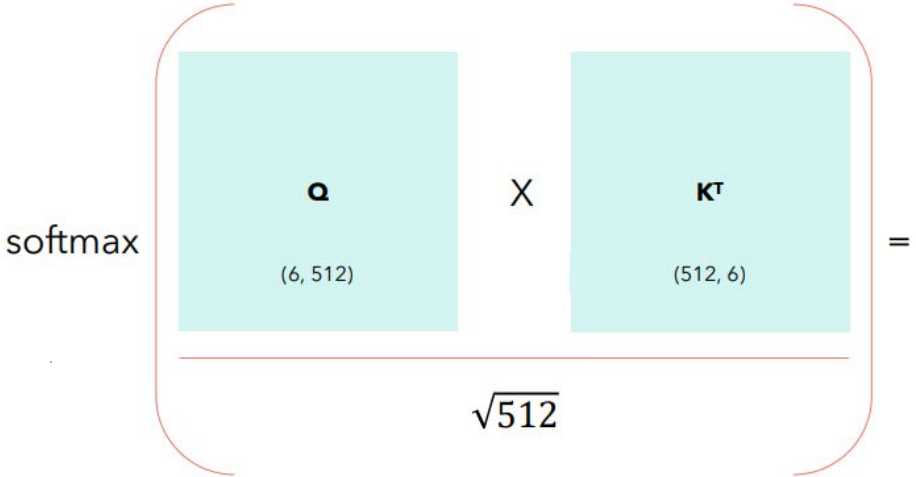
	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

(6, 6)

* all values are random.

To compute the attention score between two words, the dot product of the Query vector of one word and the Key vector of another is calculated. This score represents the relevance of the second word to the first.

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



=

	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

(6, 6)

* all values are random.

These scores are then scaled (by dividing by the square root of the vector dimension) to prevent very large values. This also helps stabilize the gradients during training.

A softmax function is applied to turn these scores into probabilities. Each row now sums to 1 — showing how much attention each word gives to others.

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.156	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

(6, 6)

X

V

(6, 512)

=

Attention

(6, 512)

Each row in this matrix captures not only the meaning (given by the embedding) or the position in the sentence (represented by the positional encodings) but also each word's interaction with other words.

Finally, we use these attention weights to combine the value vectors.

This produces the contextualized representation for each word — meaning each word's vector now includes information from other relevant words in the sentence.

While self-attention provides valuable insights into word relationships, it might not capture every possible feature of a word, such as its role as a noun, verb, or even an adjective depending on the context.
This is where *multi-head attention* comes in, as an extension of self-attention.

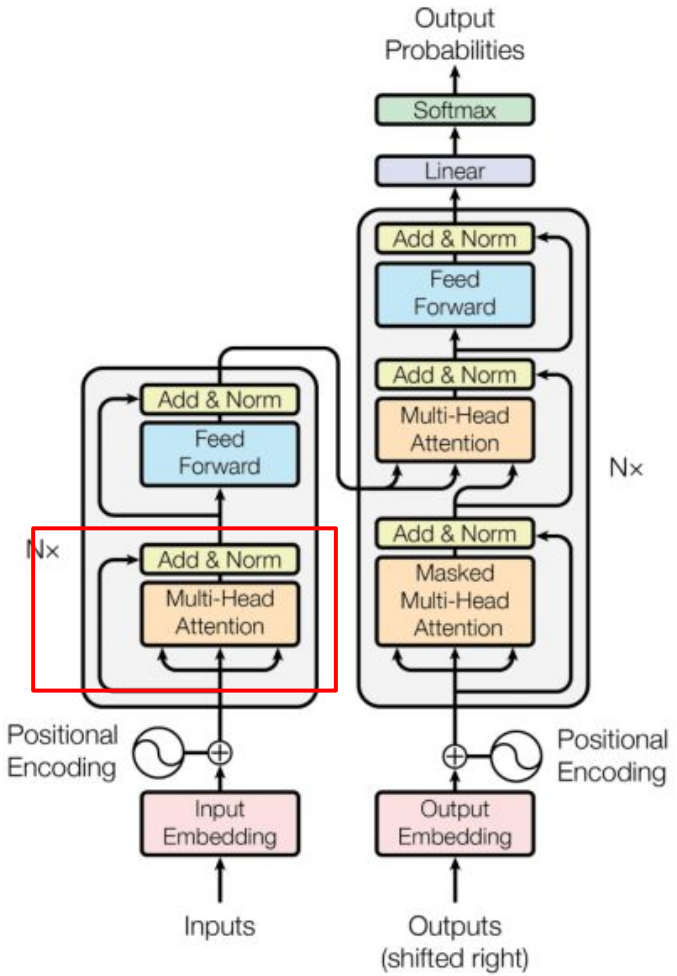
Transformer Architecture

Multi-head Attention

An extension of self-attention

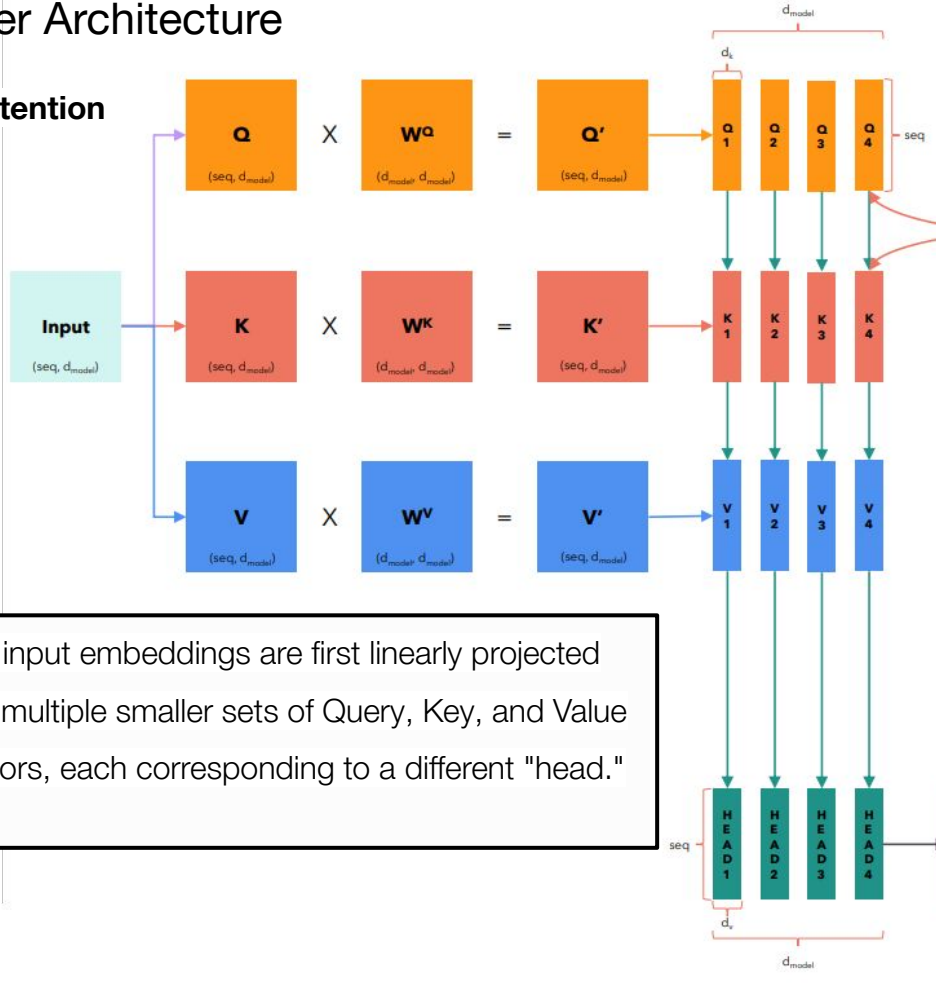
Allows the model to look at the input from different perspectives by splitting the attention mechanism into multiple parallel "heads."

Each head processes the sentence independently, focusing on different aspects or relationships.

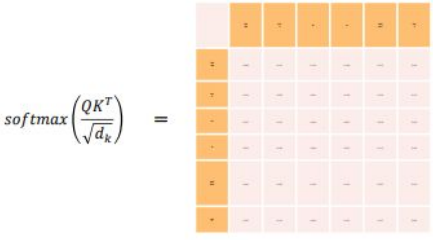


Transformer Architecture

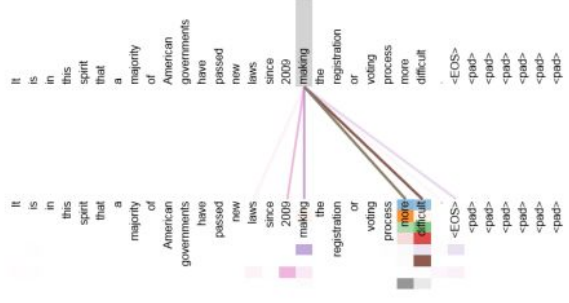
Multi-head Attention



$$softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) =$$



Attention Visualizations

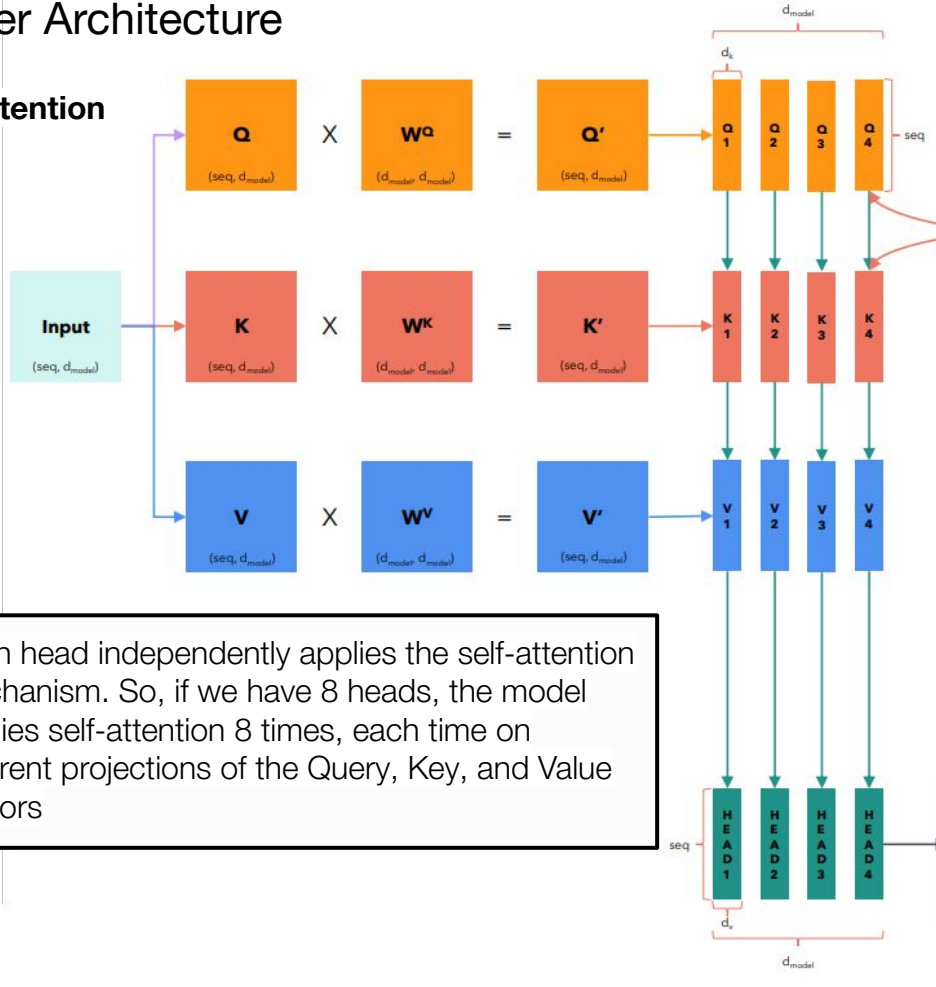


The input embeddings are first linearly projected into multiple smaller sets of Query, Key, and Value vectors, each corresponding to a different "head."

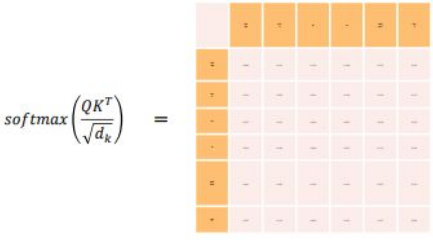
$$MultiHead(Q,K,V) = Concat(head_1 \ldots head_h)W^O$$

Transformer Architecture

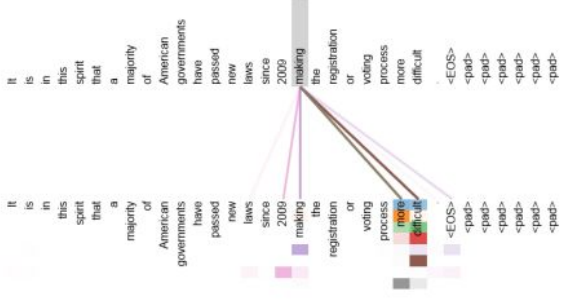
Multi-head Attention



$$softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) =$$



Attention Visualizations

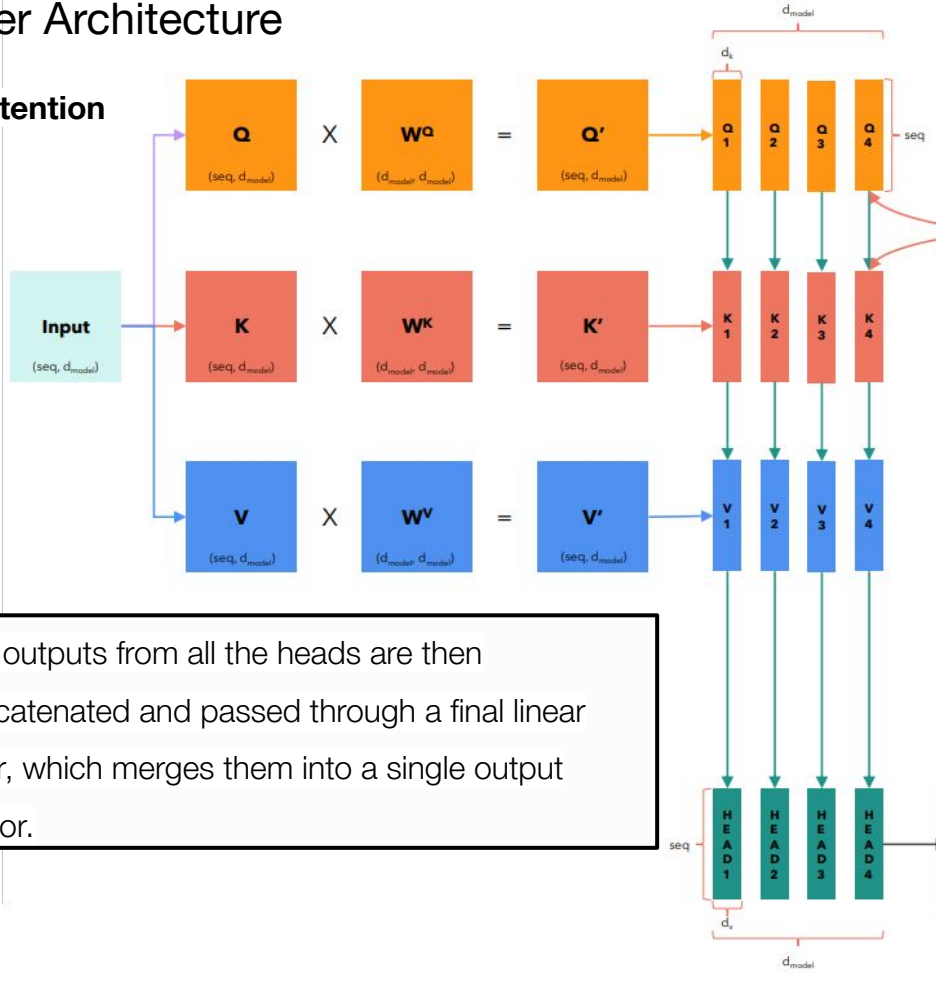


Each head independently applies the self-attention mechanism. So, if we have 8 heads, the model applies self-attention 8 times, each time on different projections of the Query, Key, and Value vectors

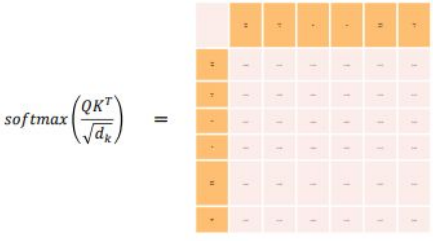
$$MultiHead(Q,K,V) = Concat(head_1 \ldots head_h)W^O$$

Transformer Architecture

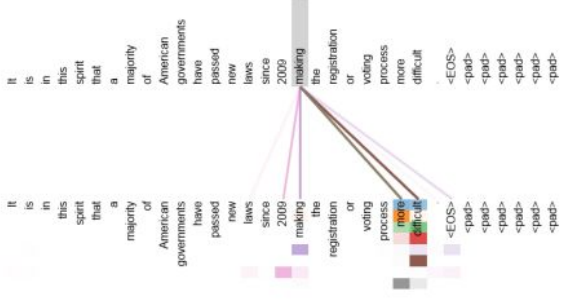
Multi-head Attention



$$softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) =$$



Attention Visualizations



The outputs from all the heads are then concatenated and passed through a final linear layer, which merges them into a single output vector.

$$MultiHead(Q,K,V) = Concat(head_1 \ldots head_h)W^O$$

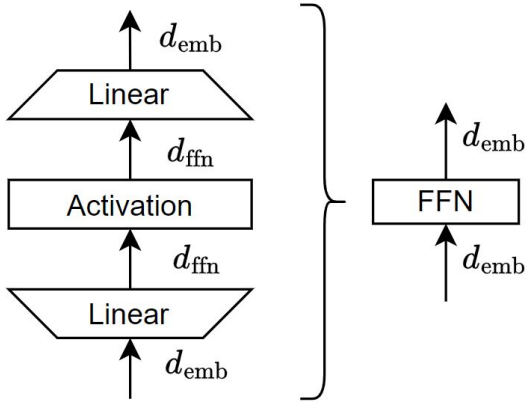
Transformer Architecture

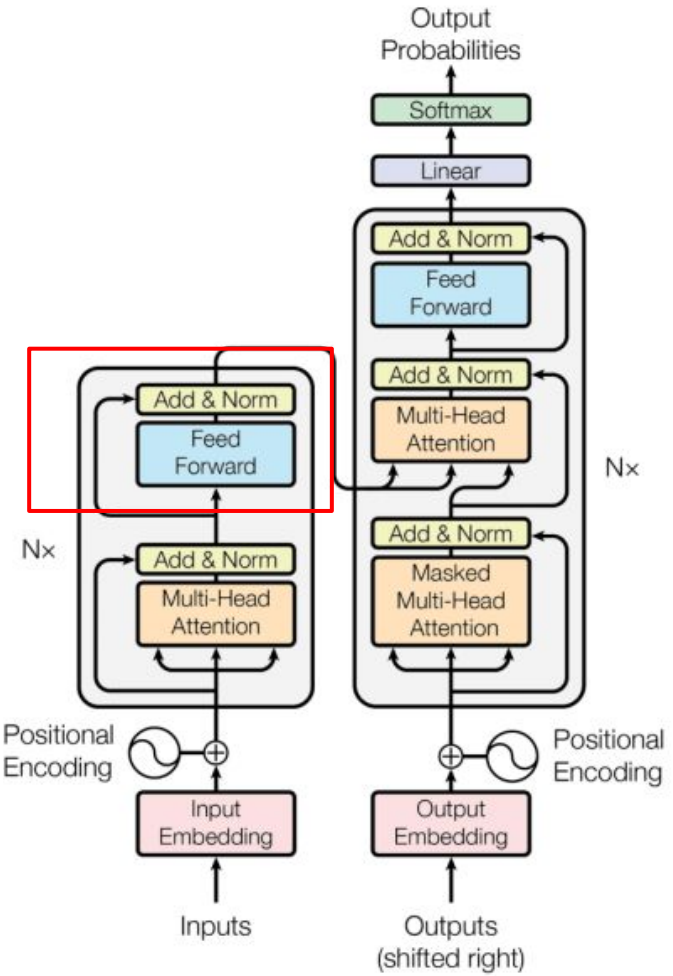
Feed-Forward Network

In each layer of the Transformer, a Feed-Forward Network (FFN) is applied to **every word independently**.

It consists of two linear transformations with a ReLU activation in between, which introduces non-linearity and helps the model capture complex patterns.

- First Linear Transformation: The input word representation (often 512 dimensions in size) is first passed through a linear transformation, which projects it to a higher-dimensional space (typically 2048 dimensions).
- ReLU Activation: After the first transformation, a ReLU activation is applied to introduce non-linearity. This non-linear step allows the model to learn more complex patterns and relationships in the data.
- Second Linear Transformation: The ReLU output is then passed through a second linear transformation, which projects it back to the original dimensionality (usually 512).





Transformer Architecture

Add & Norm

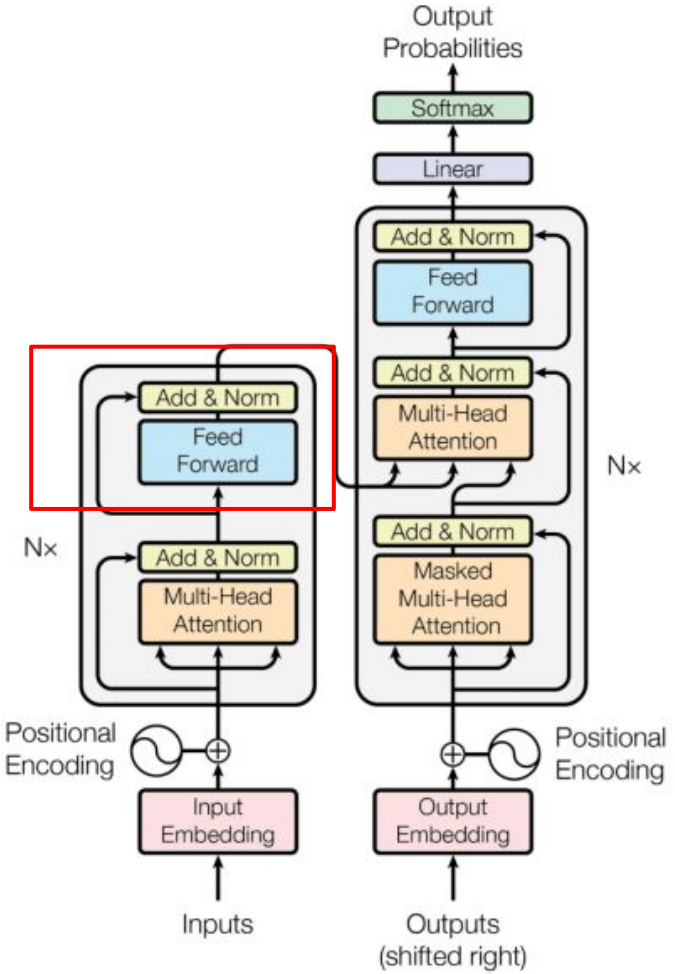
Combination of residual connection and layer normalization applied to a sublayer (Multi-Head Attention or Feed Forward)

Residual Connection (Add)

Instead of feeding only the output of the layer into the next layer, input is added to the output

$$y = x + Sublayer(x)$$

This allows the network to preserve the original information and helps gradients flow more easily during backpropagation (mitigating the vanishing gradient problem)



Transformer Architecture

Add & Norm

Combination of residual connection and layer normalization applied to a sublayer (Multi-Head Attention or Feed Forward)

Normalization

After adding we apply layer normalization to stabilize and scale the vector

$$\text{Output} = LayerNorm(x + Sublayer(x))$$

This standardizes the inputs to a layer so that they have zero mean and unit variance.

```
Input: [1, 10, 100, 1000]
```

The large differences makes it hard for the layer to learn because weights have to adjust to different scales.

Zero mean: the average becomes 0

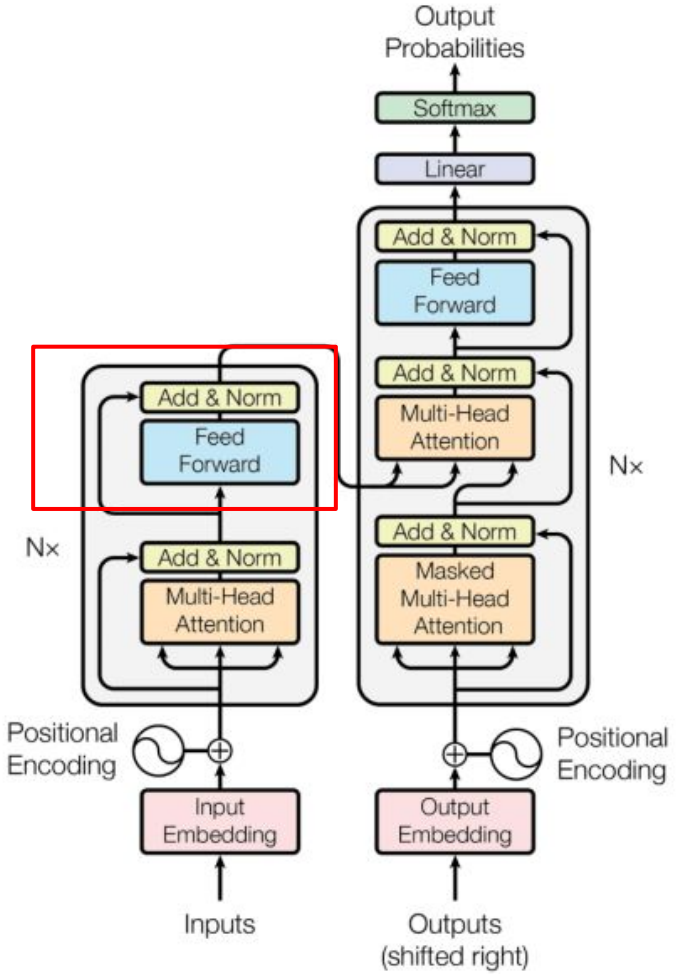
```
[1, 2, 3]  the mean is 2. Subtract it.
[-1, 0, 1] the values are centered around zero.
```

Unit variance: the spread is scaled so the variance is 1

```
[1, 0, 1] squared deviations.
(1+0+1)/3 = 0.667 close to 1 after scaling.
```

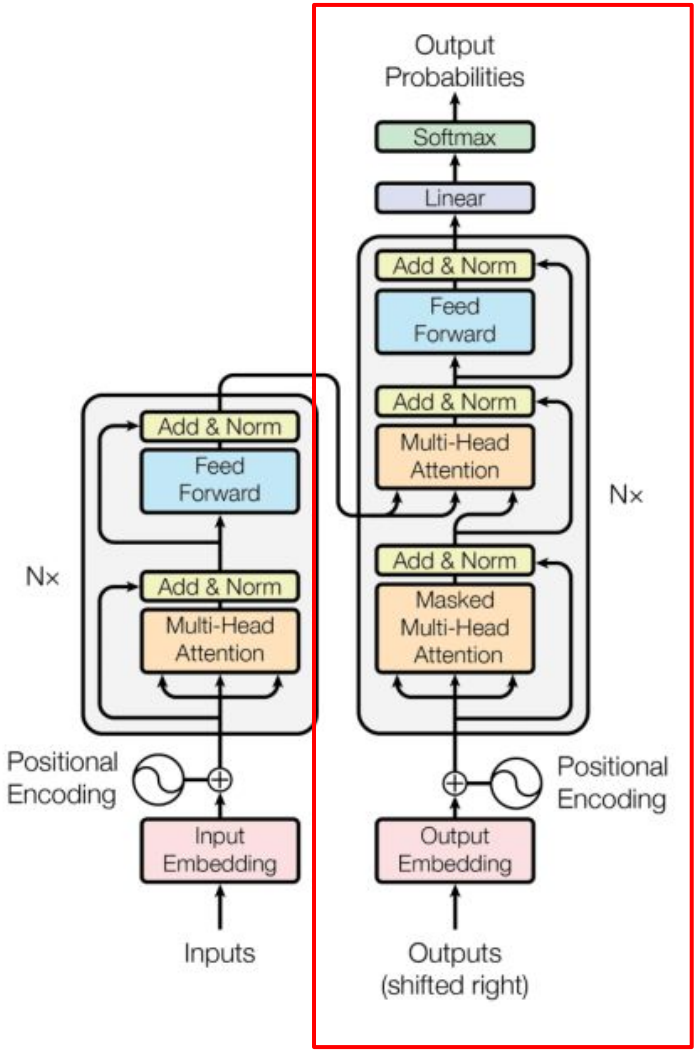
Neural networks work best when inputs are on a similar scale.

Variance = 1 ensures that no feature is too big or too small, which keeps learning stable and efficient.



Transformer Architecture: Decoder

The decoder in a transformer model consists of several layers, each with three main components that work together to generate output sequences based on the encoded input and previously generated tokens.



Transformer Architecture: Decoder

The decoder in a transformer model consists of several layers, each with three main components that work together to generate output sequences based on the encoded input and previously generated tokens.

Masked Multi-Head Self-Attention

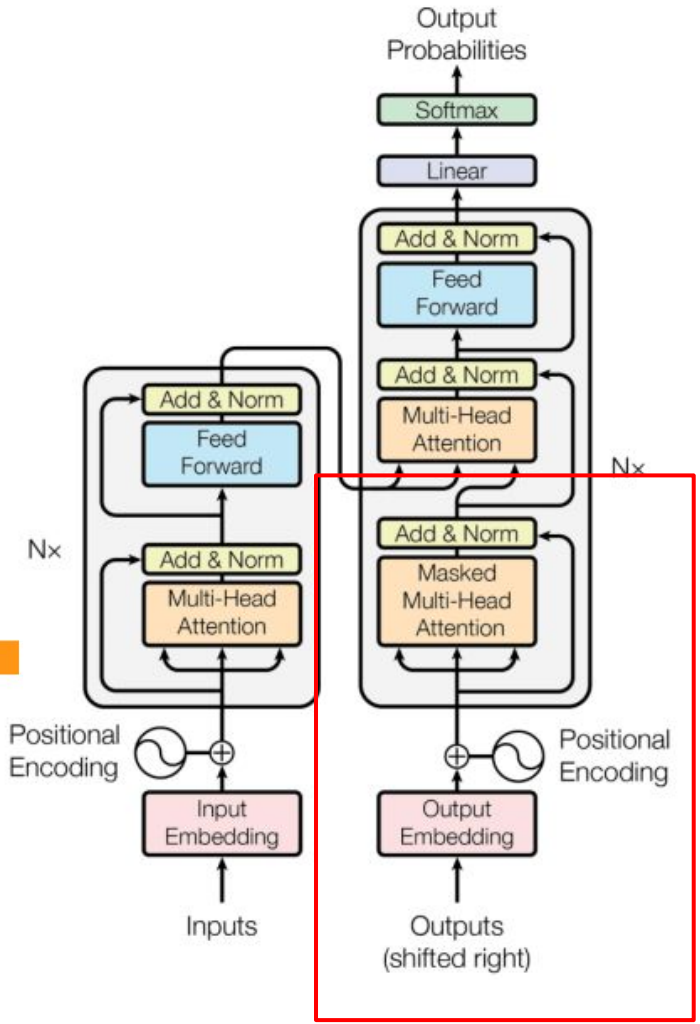
The first attention mechanism in the decoder. Similar to self-attention in the encoder but with masking.

Future positions in the sequence are masked to ensure that each position in the output can only attend to previous positions.

E.g




```
While generating "IS," the model would only be allowed to
attend to "YOUR" and "CAT," but not to "A," "LOVELY," or
the second "CAT."
```



Transformer Architecture: Decoder

The decoder in a transformer model consists of several layers, each with three main components that work together to generate output sequences based on the encoded input and previously generated tokens.

Multi-Head Cross-Attention

"The next sub-layer performs multi-head attention over the output of the encoder stack."

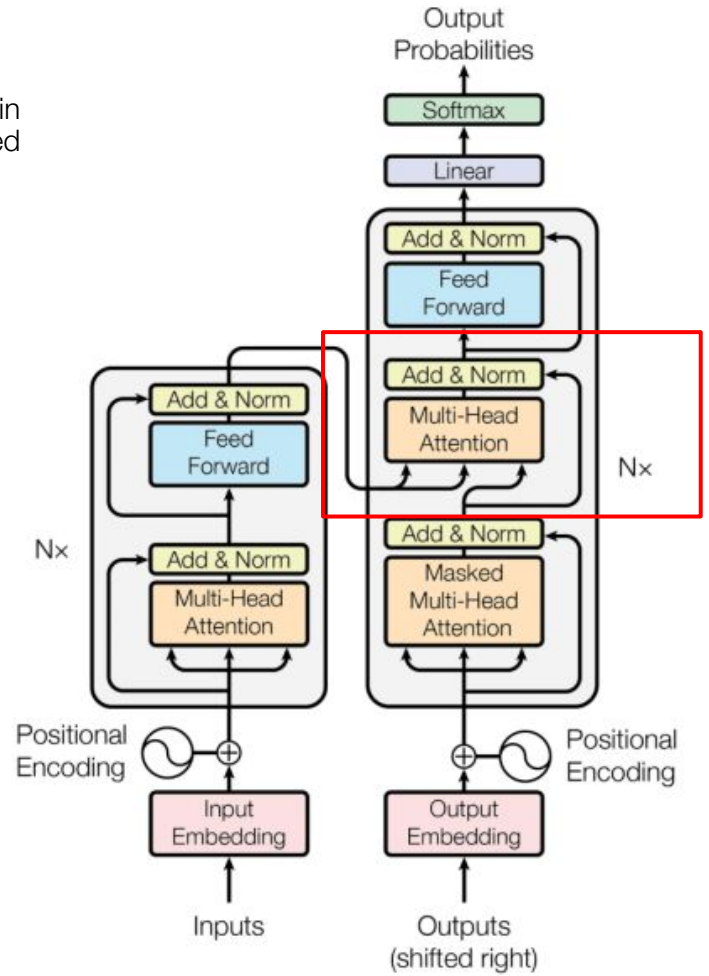
Connects the encoders output to the decoder.

Allows the decoder to attend to the full context of the input sequence from the encoder while generating each word in the output sequence.

In this layer:

- The Query (Q) comes from the decoder's output (after masked self-attention).
- The Key (K) and Value (V) come from the encoder's output.

Cross-attention to distinguish from self-attention because it attends to a different sequence which is the encoder's output rather than the decoder's output.



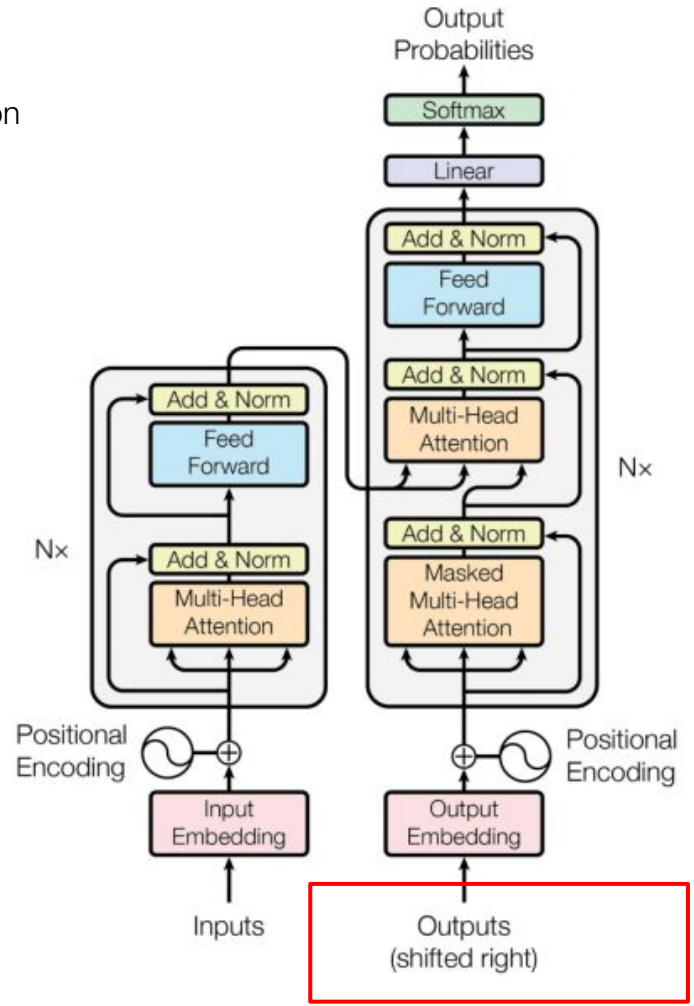
Transformer Architecture: Decoder

At the start of the training, the input to the decoder masked multi-head attention comes from target sequence shifted right.

Target sequence: `your cat is a lovely cat </s>`

Decoder input: `<s> your cat is a lovely cat`

Decoder Input	Decoder Predicts (Target)
`<s>`	your
`<s>, your`	cat
`<s>, your, cat`	is
`<s>, your, cat, is`	a
`<s>, your, cat, is, a`	lovely
`<s>, your, cat, is, a, lovely`	cat
`<s>, your, cat, is, a, lovely, cat`	</s>



Transformer Architecture: Decoder

Linear & Softmax

The decoder's output is passed through a linear transformation and a softmax layer to produce a probability distribution over the entire vocabulary. The word with the highest probability is selected as the next word in the sequence

$$\hat{\mathbf{y}}^{\,t} = [p_1, p_2, \ldots, p_V]$$

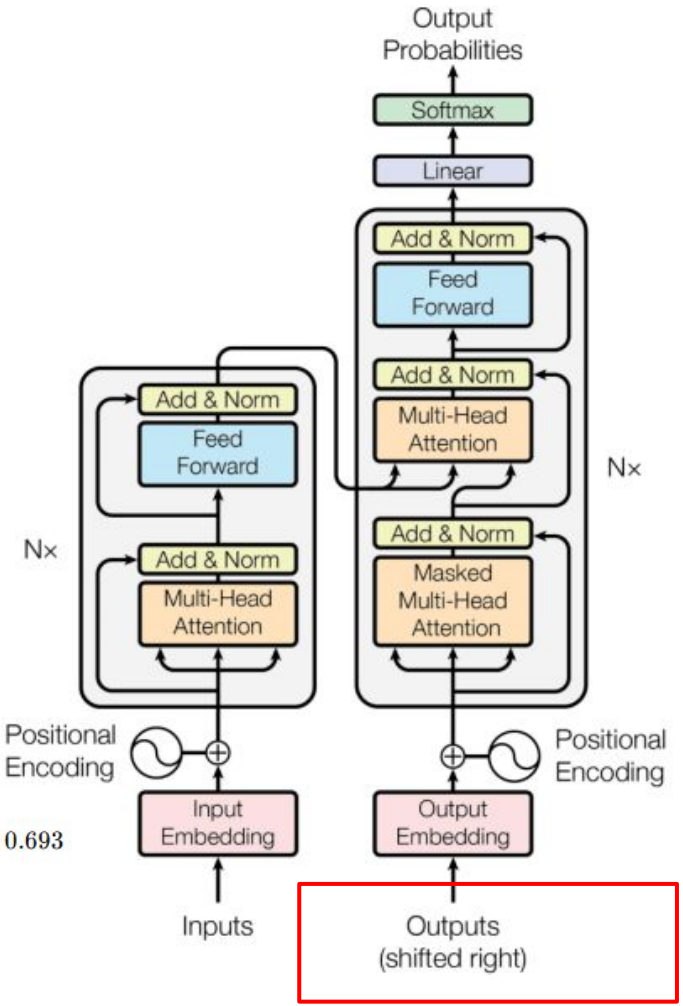
In the model, a true token is represented as a one-hot vector.

$$\mathbf{y}^{\,t} = [y_1, y_2, \ldots, y_V], \quad y_i = \begin{cases} 1 & \text{if token } i \text{ is correct at timestep } t \\ 0 & \text{otherwise} \end{cases}$$

```
probability_vector = [0.1,0.2,0.5,0.1,0.1]
target_vector = [0,0,1,0,0]
```

$$\mathcal{L}_t = -\sum_{i=1}^{5} y_i \log \hat{y}_i = -(0 \cdot \log 0.1 + 0 \cdot \log 0.2 + 1 \cdot \log 0.5 + 0 \cdot \log 0.1 + 0 \cdot \log 0.1) = -\log 0.5 \approx 0.693$$

Loss is small if correct token has high probability, large if low



Resources

1. Vaswani et al., 2017 – *Attention Is All You Need*
   The original Transformer paper introducing self-attention.

2. MIT Deep Learning Lectures
   Deep sequence modeling and Transformer architecture. https://introtodeeplearning.com/

3. “Transformers Simplified: A Guide to Attention is All You Need” – Moiz Asghar
   https://www.linkedin.com/pulse/transformers-simplified-guide-attention-all-you-need-moiz-asghar-zdvmc/

4. Google Gemini
   Recent research and developments in large language models from Google.

5. ChatGPT (OpenAI)
   Interactive AI assistant for understanding Transformers, NLP concepts, and more.
   https://chat.openai.com